

RAPPORT DE PROJET GUERLÉDAN

RAMI DIAB, BRUNO HUNTZINGER,
HIPPOLYTE LAFRAD, MATHIS LE SCOËZEC

PROJET INTERCEPTEUR

5 mars 2026

ENST2



IP PARIS



Remerciements :

Avant de poursuivre, nous souhaitons remercier M. Ronan Douguet, M. Yvan Eustache et M. Dominique Heller, enseignants chercheurs à l'Université de Bretagne Sud pour nous avoir encadrés tout au long de ce projet. Nous remercions également M. Fabrice Le Bars, enseignant chercheur à l'ENSTA pour ses précieux conseils.

Table des matières

Introduction	3
I Présentation l'USV IM1800 et du rover R1	4
I.1 L'USV IM1800 et son hardware	4
I.2 Le rover R1	8
II Algorithmes d'interception	9
II.1 Contrôleur Proportionnel-Intégral-Dérivé (PID)	9
II.2 Contrôleur Prédicatif pour Modèles non-Linéaires (NMPC)	11
II.3 Guidage par asservissement visuel	17
III Présentation de l'interface homme-machine (IHM)	33
IV Validation de nos algorithmes sur le rover R1	36
V Validation de nos algorithmes sur l'intercepteur M1800	38
Conclusion	39

Introduction

L'objectif de ce présent rapport est de dresser un bilan de nos travaux menés dans le cadre de notre projet Guerlédan. Le projet Intercepteur sur lequel nous avons travaillé avait pour objectif de permettre à un véhicule de surface autonome (USV) de réaliser l'interception physique d'un autre USV ou de toute cible navale mobile pénétrant dans une zone maritime sensible (champ d'éoliennes en mer, base militaire, etc...).

Pour ce faire, nous avons la possibilité de récupérer les données de plusieurs capteurs installés à bord de notre USV : un radar, une caméra et un lidar. Nous devons ensuite exploiter ces données pour détecter la présence d'une éventuelle menace dans la zone que nous étions chargés de défendre. Si nous repérions qu'un USV était présent à l'intérieur de cette zone, notre drone de surface était chargé de l'intercepter physiquement à l'aide de programmes de guidage que nous avions pour objectif d'implémenter. Dans le but de ne pas détériorer l'USV qui nous servait de cible, nous nous sommes contentés de maintenir notre intercepteur à quelques mètres de notre cible et de la suivre, quelles que soient les évolutions de sa trajectoire.

Tout au long de ce projet, nous avons été accompagnés par une équipe d'enseignants chercheurs de l'Université de Bretagne Sud (UBS) qui nous ont fourni l'essentiel des moyens nécessaires à sa réalisation.

Avant d'aborder la présentation des différentes parties de notre rapport, il également à noter que nous n'avons pas accès à notre USV entre les deux sessions à Guerlédan. Nous avons donc validés nos différents programmes sur un robot terrestre (rover) dont les équations cinétiques sont similaires .

Ce présent rapport s'articule en six parties. La première se charge de présenter les différents robots dont nous nous sommes servis au cours du projet. La deuxième partie aborde ensuite les différents algorithmes retenus pour réaliser l'interception de l'USV cible, suivie d'une troisième partie expliquant comment ces algorithmes ont pu être validés par des simulations. Une quatrième partie vient, en parallèle, détailler la réalisation de l'interface homme-machine (IHM) ayant été implémentée afin de rendre plus aisée pour les opérateurs la réalisation des différentes missions de l'intercepteur. Pour finir, la cinquième et la sixième partie viennent présenter les différentes expériences réalisées sur le rover, puis sur l'intercepteur lui-même au lac de Guerlédan, expériences grâce auxquelles nous avons pu valider nos algorithmes d'interception.

I Présentation l'USV IM1800 et du rover R1

I.1 L'USV IM1800 et son hardware

Capteurs fonctionnels

Le robot marin de surface (Unmanned Surface Vehicle - USV) que nous avons utilisé en temps qu'intercepteur et celui que nous avons utilisé en tant que cible sont tous les deux des drones de surface multi-rôles M1800 d'IMSolutions [Figure 1]. Ils possèdent un modem Wi-Fi et un modem LTE pour permettre de se connecter à un réseau Wi-Fi ou à un réseau 4G et peuvent également être contrôlés par télécommande via un module de réception des signaux radio.



FIGURE 1 – L'intercepteur sur le lac de Guerlédan

L'architecture électronique des deux USVs se base sur une carte Hexcube Black sur laquelle est installée le logiciel ArduPilot et une carte électronique Jetson Orin [Figure 2]. Sur la carte Hexcube Black est branché un récepteur GNSS Ellipse D de SBG Systems faisant également office de centrale inertielle (IMU). L'Hexcube Black est également chargée de contrôler les moteurs. Elle communique les informations du récepteur GNSS Ellipse D à la Jetson Orin en utilisant le protocole MavLink tandis que la Jetson, elle, lui transmet les consignes, aussi bien pour contrôler la vitesse des deux moteurs que pour suivre un point en autonomie. Dans notre cas, nous utilisons la

surcouche MavROS, qui permet de communiquer directement en topic ROS2 avec l'ArduPilot.

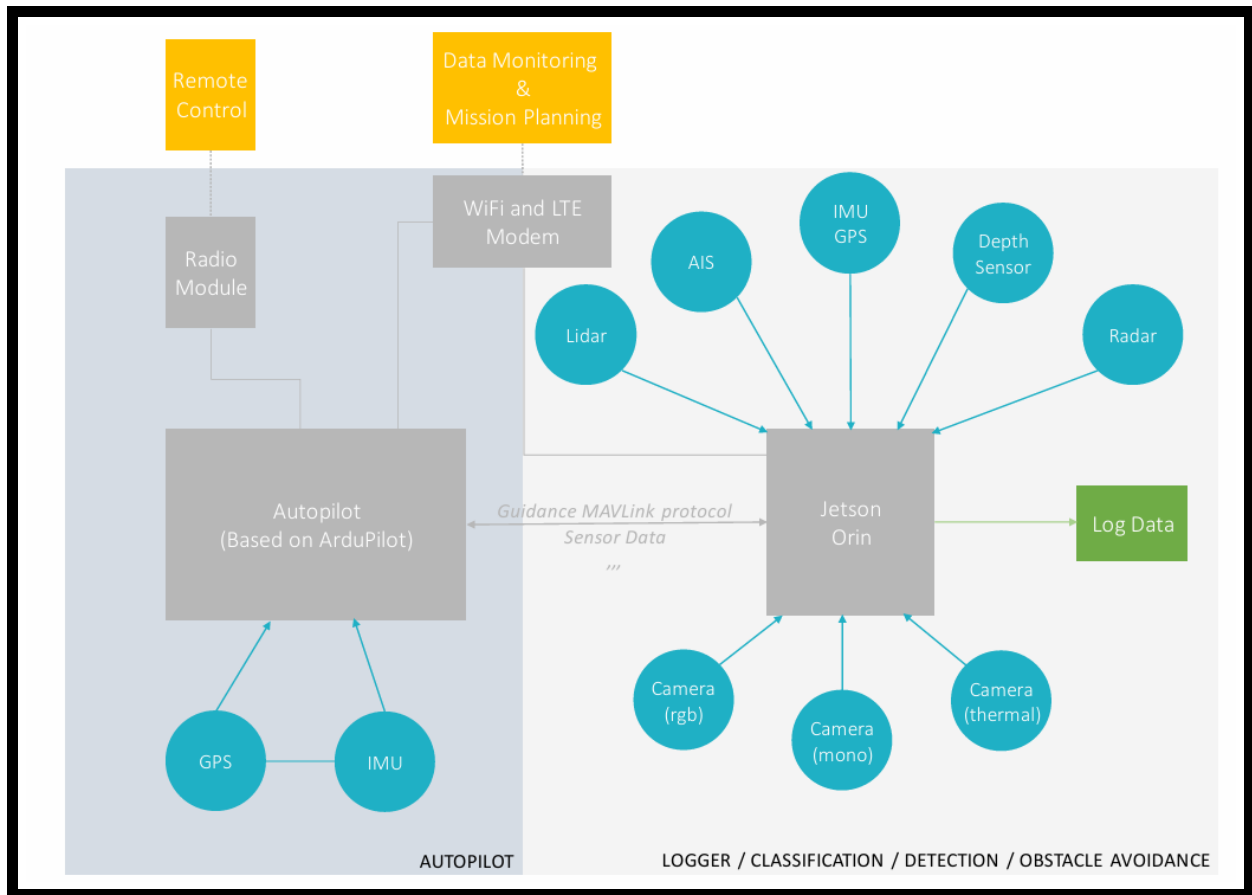


FIGURE 2 – Architecture de contrôle et architecture capteur du robot intercepteur (source : UBS)

Du côté de l'intercepteur, de nombreux capteurs viennent également enrichir sa connaissance de l'environnement extérieur. Tout d'abord, un récepteur AIS (Automatic Identification System) lui permet de localiser les différents navires présents autour de lui. Une caméra monochrome MV-CA020-20-GM d'Hikrobot [Figure 3], une caméra RGB acA1920-50gc de BASLER et une caméra thermique FLIR ADK de FLIR OEM permettent aux opérateurs du drone de disposer d'images en provenance du drone, de jour comme de nuit, leur permettant de mieux apprécier la situation autour de lui. Un lidar OS-1-64 d'OUSTER de 120 m de portée [Figure 4] et un radar ARS-408 de Continental de 250 m de portée viennent compléter cette panoplie d'équipements.



FIGURE 3 – La caméra monochrome MV-CA020-20-GM d'Hikrobot



FIGURE 4 – Le LIDAR OS-1-64 d'OUSTER

Intégration du radar dans l'architecture logicielle

Le radar ARS-408 [Figure 5] est un capteur majoritairement utilisé dans le milieu de l'automobile. Peu coûteux (moins de 100€ aujourd'hui) comparé à d'autres capteurs similaires comme le lidar, il embarque des algorithmes de traitements avancés et ne se contente pas de renvoyer des données brutes. Il communique grâce au protocole CAN avec la carte Jetson. Il a une portée de près de 200m moyennant le respect de certaines conditions.



FIGURE 5 – Le radar ARS-408 de Continental

En effet, l'ARS-408 possède deux modes : le mode "Cluster" ou le mode "Object". Dans le premier, le traitement nous donnera en sortie du radar des clusters - des amas de points étant identifiés comme faisant partie d'une seule entité. Le mode Object nous renvoie lui directement les objets que pense avoir identifié le radar. Ces objets ont plusieurs classes (humain, camion...) et sont suivis automatiquement par le radar qui leur attribue un identifiant. Cependant ces classes sont optimisées pour le secteur de l'automobile, ce qui fait que les gros objets vont ressortir comme une multitude de petits objets. Les deux modes nous permettent d'obtenir l'angle (azimut), la taille, la position, la distance, la vitesse et l'accélération de l'objet/cluster identifié par le

radar dans le repère de ce dernier. Nous avons également accès à des informations plus spécifiques comme la puissance de retour du signal. Dans nos expérimentations, nous avons privilégié le mode "Object" car il s'adapte bien au suivi d'un USV de moins de 2m.

Le radar avait déjà été testé par l'équipe de l'UBS, mais les drivers qui étaient fonctionnels avaient été conçus pour ROS1 et non ROS2. Certaines bibliothèques compatibles avec ROS2 ont été créées par des utilisateurs du radar, mais nous ne sommes jamais parvenus à les faire fonctionner. Nous avons donc dû coder nous même un driver ROS2 pour le radar. Cette tâche fut relativement fastidieuse car il a fallu coder toutes les fonctions de décodage des trames CAN. Nous nous sommes donc concentré sur les trames qui nous seraient utiles ce qui fait que notre driver ne couvre pas l'entièreté des fonctions du radar.

Nous avons également implémenté un nœud ROS2 qui publie des messages interprétables par RVIZ2 - un logiciel de visualisation intégré à ROS2 [Figures 6 et 7].

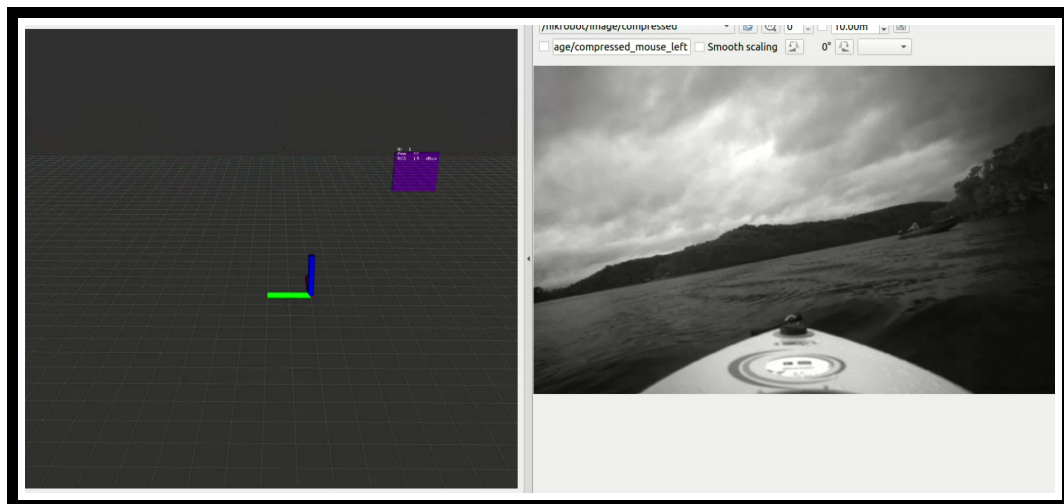


FIGURE 6 – Vue radar vs caméra, on voit clairement la barge dans le repère radar.

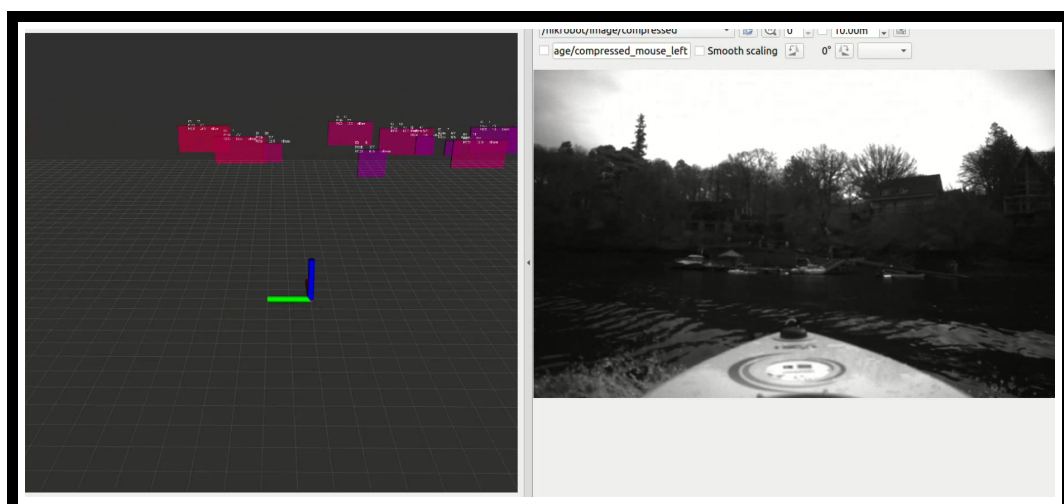


FIGURE 7 – Ici on voit une multitude d’objets, car les filtres sont mal configurés pour les grands objets.

Comme nous pouvons obtenir des informations de vitesses et de positions, nous pouvons donc utiliser le radar pour estimer l’état de la cible et donc prédire son état futur. Ces informations nous permettraient de développer des algorithmes d’interception bien plus efficaces. Mais par manque de temps d’expérimentation, nous n’avons pas pu tester cette fonctionnalité.

Une amélioration future serait de fusionner les données caméras avec les données radar, pour permettre une meilleure identification et un meilleur tracking de la cible.

I.2 Le rover R1

N’ayant pas accès à l’USV entre nos deux semaines d’expérimentation à Guerlédan, le rover R1 d’AION Robotics [Figure 8] nous a été prêté par l’UBS. Ce robot a une architecture logicielle très similaire à celle de notre USV : il fonctionne également avec le tandem Jetson Orin / Hexcube Black, dispose aussi du récepteur GNSS Ellipse D, se meut grâce à une motorisation différentielle et est également commandé grâce à MAVROS. Il est cependant équipé de bien moins de capteurs et ne permet pas de valider l’intégralité des algorithmes de contrôle que nous avons mis en place. Il nous a tout de même permis de prendre en main MAVROS et de tester certains contrôles basiques. De plus, nous avons également pu intégrer certains capteurs sur ce rover, notamment la caméra acA1920-50gc et le radar ARS-408.

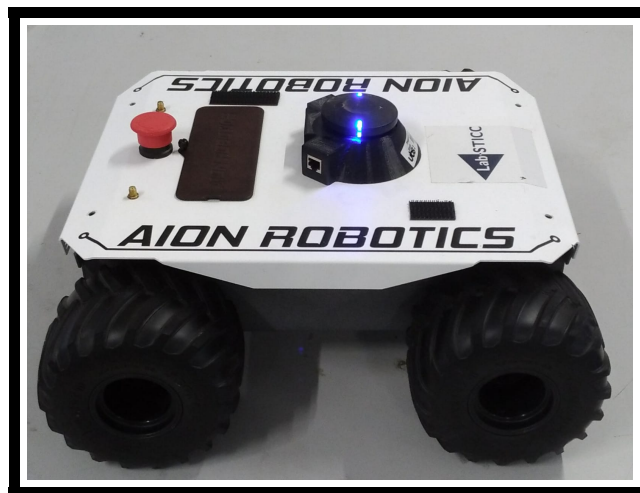


FIGURE 8 – Le rover R1 d’AION Robotics

II Algorithmes d'interception

II.1 Contrôleur Proportionnel-Intégral-Dérivé (PID)

Le premier contrôleur que nous avons décidé de mettre en place est un contrôleur PID. Ce dernier est très simple mais reste efficace si les gains proportionnel, dérivé et intégral sont bien paramétrés. Ce contrôleur nous a notamment permis de tester notre méthode de guidage par asservissement visuel avant de passer à un contrôleur plus élaboré. Son principe est expliqué par le schéma ci-dessous [9], l'objectif étant, de rendre la sortie $S(s)$ égale à consigne $E(s)$.

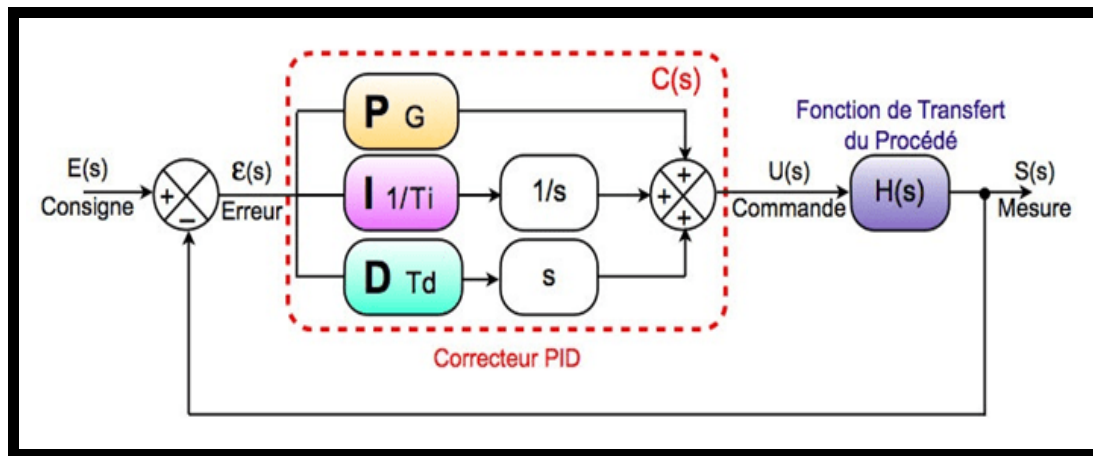


FIGURE 9 – Explication du fonctionnement du contrôleur PID

La régulation Proportionnelle – Intégrale – Dérivée est probablement la régulation la plus utilisée en robotique mobile. Dans notre cas, elle permet de corriger le cap de l'intercepteur afin qu'il suive une valeur de consigne dynamique calculée par les algorithmes de détection de la cible. Le principe général du contrôleur PID repose sur la correction de l'erreur entre la consigne et la valeur mesurée du système. Le régulateur agit alors sur les entrées du système pour ajuster son comportement et réduire cette erreur. Le PID combine trois actions complémentaires : une action proportionnelle, une action intégrale et une action dérivée. Chacune de ces actions possède un paramètre de réglage spécifique qui influence le comportement dynamique du système.

Le paramètre proportionnel, noté K_p , produit une action proportionnelle à l'erreur instantanée. Plus l'erreur est grande, plus la correction appliquée est importante. Cette action permet de réagir rapidement aux écarts, améliorant par là la rapidité de réponse du système. Toutefois, si K_p est trop élevé, le système peut devenir instable ou être sujet à des oscillations importantes. À l'inverse, s'il est trop faible, la réponse sera lente et l'erreur résiduelle risque de persister.

Le paramètre intégral, noté K_i , agit sur l'accumulation de l'erreur dans le temps. Il permet de corriger l'erreur statique, c'est-à-dire l'écart permanent entre la consigne et la sortie du système. Mathématiquement, l'action intégrale correspond à l'intégrale de l'erreur par rapport au temps. Elle améliore donc la précision à long terme.

Cependant, une valeur excessive de K_i peut entraîner des dépassements importants et des oscillations prolongées, voire un phénomène appelé « intégration excessive » (ou windup), qui dégrade la stabilité.

Le paramètre dérivé, noté K_d , est lié à la variation de l'erreur dans le temps, c'est-à-dire à sa dérivée. Il anticipe l'évolution de l'erreur en réagissant à sa vitesse de variation. Cette action a pour effet d'amortir le système, de réduire les dépassements et d'améliorer la stabilité dynamique. Néanmoins, l'action dérivée est sensible au bruit de mesure, la dérivation amplifiant les variations rapides et les perturbations.

Le système simulé comprend deux entités. Le robot intercepteur est modélisé comme un double intégrateur en 2D, représentant un objet à masse implicite se déplaçant dans un plan. Son état est décrit par sa position (x, y) et sa vitesse (v_x, v_y) , et il est commandé par des accélérations (a_x, a_y) selon les équations d'Euler explicite : la nouvelle position est obtenue en ajoutant à la position actuelle la vitesse multipliée par le pas de temps Δt , et la nouvelle vitesse en ajoutant à la vitesse actuelle l'accélération multipliée par Δt . Ce modèle simule un véhicule simple capable d'accélérer dans n'importe quelle direction pour atteindre sa cible. Le robot cible adopte quant à lui un comportement imprévisible : il change son accélération de façon aléatoire toutes les 2 secondes (paramètre ajustable), avec une vitesse plafonnée à 50 unités pour éviter des mouvements trop rapides, et rebondit sur les bords de l'espace de simulation avec une perte d'énergie modélisée par un coefficient de restitution de 0.8. Cette approche génère un mouvement réaliste mais difficile à prévoir.

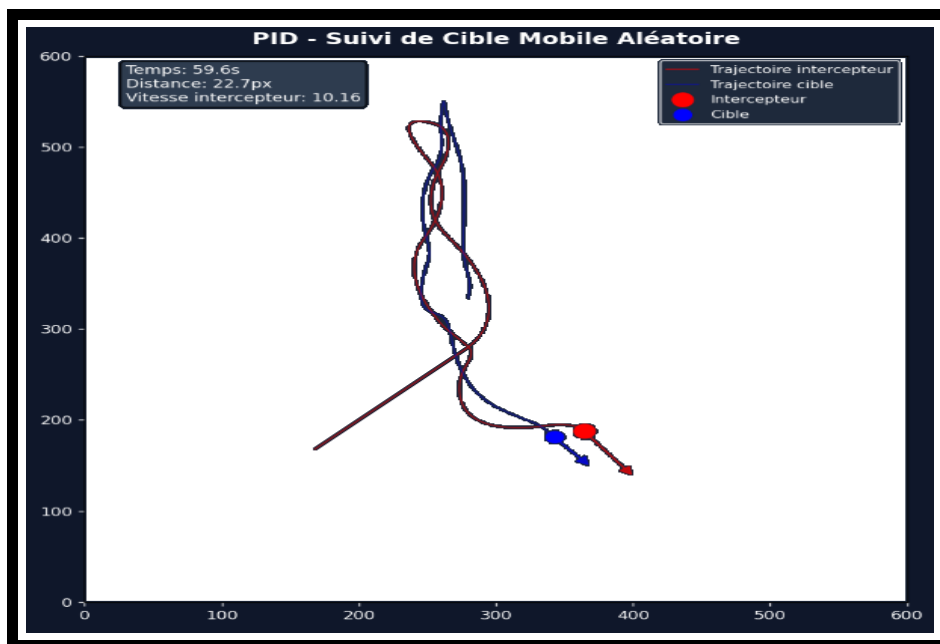


FIGURE 10 – Simulation d'interception avec un contrôleur PID

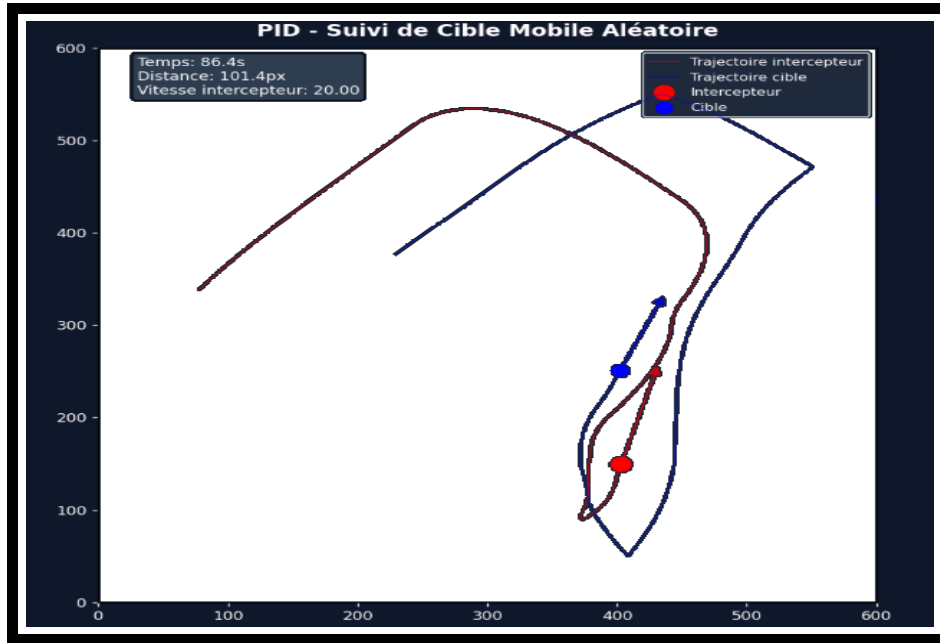


FIGURE 11 – Autre simulation d’interception avec un contrôleur PID

Sur les deux images ci-dessus [10], on peut observer que le robot intercepteur (rouge) parvient à poursuivre et rattraper sa cible (bleue), même si il peine ensuite à se maintenir sur sa position. Pour les premiers tests, ce contrôleur s’est donc révélé être suffisant.

Le principal avantage du contrôleur PID réside dans la simplicité de sa mise en œuvre et son universalité. Toutefois, il présente également des limites : son réglage peut être délicat, surtout pour des systèmes complexes, non linéaires ou à forte dynamique variable. Dans notre cas, un P.I.D bien réglé nous suffisait pour valider notre algorithme de guidage, mais pour permettre une chasse plus dynamique et intelligente, d’autres algorithmes comme le NMPC se montreront plus efficaces.

II.2 Contrôleur Prédicatif pour Modèles non-Linéaires (NMPC)

Afin d’améliorer les performances de l’intercepteur pour la traque de cibles à trajectoire non linéaire, nous avons ensuite exploré la piste du Contrôleur Prédicatif de Modèle Non-Linéaire (en anglais Non-Linear Model Predictive Control (NMPC)). Bien qu’il n’ait pas pu être déployé directement sur l’USV intercepteur, il a été validé sur simulateur via Unity 6, comme il sera détaillé dans la partie Simulation.

Le principe du NMPC repose sur la capacité du robot à prédire les futurs déplacements de la cible sur un horizon temporel fini, puis à calculer à chaque instant la séquence de commandes optimale permettant de l’intercepter tout en respectant ses propres contraintes mécaniques et dynamiques. Le contrôleur opère en boucle fermée selon quatre étapes répétées à chaque itérations de la boucle de contrôle : mesure de l’état actuel du système, prédiction des états futurs sur un horizon de N itérations, optimisation de la séquence de commandes par minimisation d’une fonction de coût sous contraintes, et enfin application de la première commande de la

séquence optimale. Le cycle est ensuite réitéré, implémentant ainsi un horizon glissant qui permet d'anticiper la trajectoire future de la cible et d'adapter continuellement la stratégie de contrôle. Il convient néanmoins de trouver un équilibre dans le degré d'anticipation : un horizon trop long nuit à la réactivité et à la fiabilité des prédictions, en particulier face à une cible changeant imprévisiblement de direction.

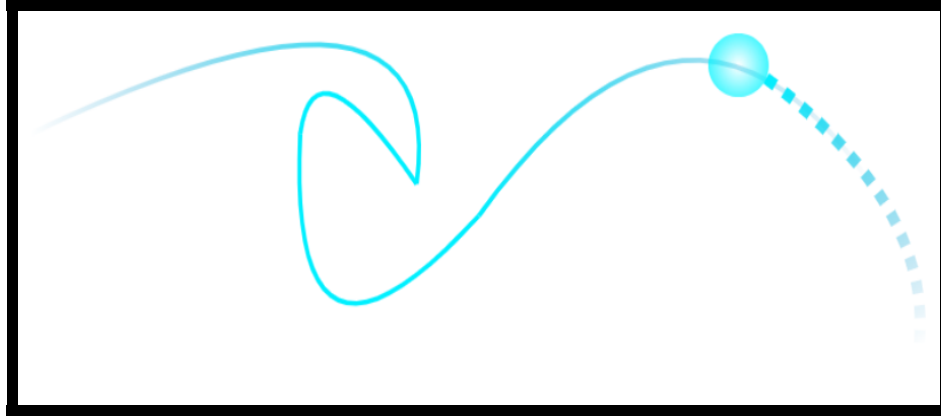


FIGURE 12 – Trajectoire de la cible prédite par le contrôleur NMPC

On suppose que la cible (représentée sur la figure [Figure 12] par un disque bleu) a déjà parcouru la trajectoire non-linéaire représentée par un trait continu bleu. L'objectif du contrôleur sera en premier lieu de prédire la trajectoire future de la cible par la cible, représentée par un trait pointillé bleu.

Trois paramètres principaux gouvernent le comportement du NMPC. L'horizon de prédiction, qui détermine le nombre de pas de temps anticipés. Un horizon court rend le robot réactif mais prédisant la trajectoire de la cible sur un court intervalle de temps, ce qui produit des trajectoires saccadées. Un horizon long, au contraire, va plutôt favoriser l'anticipation et la fluidité des mouvements, au prix d'un temps de calcul proportionnellement plus élevé et d'une fiabilité décroissante pour les prédictions lointaines temporellement, surtout en cas de mouvement imprévisible de la cible.

Le poids de contrôle, lui, régit l'agressivité du robot intercepteur : une valeur faible autorise des accélérations maximales et un comportement réactif mais brutal, tandis qu'une valeur élevée contraint le robot à des mouvements progressifs. Il constitue donc le levier principal pour adapter le contrôleur aux contraintes physiques liées à l'intercepteur et à son milieu.

Enfin, le poids d'état (qui regroupe un poids de position, un poids d'orientation et un poids de vitesse) détermine la précision du suivi : un poids faible tolère un écart raisonnable avec la cible, suffisant pour une mission de surveillance à distance, tandis qu'un poids élevé pousse le robot à minimiser l'écart à tout prix en mobilisant toutes ses capacités d'accélération.

La fonction de coût adoptée traduit formellement le compromis entre ces deux objectifs antagonistes que sont la précision du suivi et l'économie d'énergie. Elle s'exprime comme suit :

$$\text{Coût_total} = \text{Poids_état} \times (\text{erreur}(\text{position}, \text{vitesse}))^2 + \text{Poids_contrôle} \times \text{accélération}^2 \quad (1)$$

Un poids d'état élevé pousse le robot intercepteur à coller à la trajectoire de la cible, quitte à recourir à de fortes accélérations, tandis qu'un poids de contrôle élevé privilégie des mouvements plus fluides au prix d'un suivi moins précis. L'utilisation de termes quadratiques garantit que les erreurs négatives sont pénalisées aussi sévèrement que leurs équivalents positifs, et que la pénalisation croît de manière non linéaire avec l'amplitude de l'erreur. Il existe donc un conflit intrinsèque entre les deux objectifs (suivre parfaitement la cible d'un côté, économiser l'énergie de l'autre) et l'optimisation du contrôleur repose essentiellement sur le choix et le réglage de ces poids.

À chaque itération, l'algorithme se déroule en quatre étapes. Il prédit d'abord la trajectoire future de la cible en supposant que celle-ci continuera sur sa lancée à vitesse constante (une extrapolation linéaire simple) et calcule ainsi sa position attendue à chaque instant de l'horizon. Il explore ensuite l'espace des commandes possibles par recherche par force brute sur une grille discrète de valeurs d'accélération (a_x, a_y) [Figure 13], balayant les valeurs entre $-x$ et $+x$ par pas de dx (x et dx sont déterminés par l'opérateur, selon qu'il souhaite disposer de plus d'options d'accélération ou qu'il privilégie une diminution de la charge de calcul de son ordinateur embarqué). Pour chaque combinaison testée, il simule l'évolution du robot intercepteur sur l'intégralité de l'horizon temporel et évalue le coût cumulé, pénalisant à chaque pas l'écart de position entre le robot et la cible prédite, l'écart de vitesse entre les deux entités, ainsi que l'effort de contrôle lié aux accélérations élevées. La commande minimisant ce coût total sur l'ensemble de l'horizon est finalement sélectionnée et appliquée pour un seul pas de temps [Figure 14], avant que le cycle ne reprenne. Ce mécanisme d'horizon glissant garantit une replanification constante en fonction de l'état actuel du système [Figure 15].

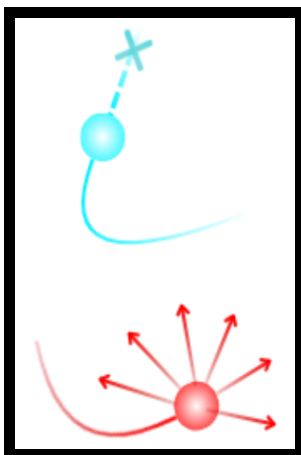


FIGURE 13 – Le contrôleur teste les différentes accélérations possibles

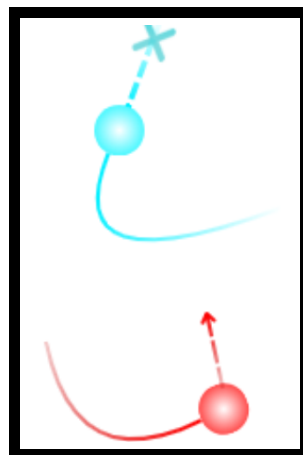


FIGURE 14 – Le contrôleur choisit l'accélération minimisant la fonction de coût total

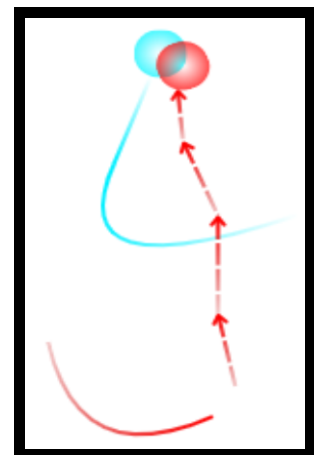


FIGURE 15 – On répète le processus à chaque itération de la boucle de contrôle

On reprend la simulation utilisée pour le PID, avec un système à 2 entités donc les modèles dynamiques sont conservés. L'interface de simulation [Figure 16] expose désormais quatre curseurs permettant d'expérimenter en temps réel : l'horizon de prédiction, réglable de 5 à 40 pas, le poids de contrôle, compris entre 0.01 et 1.0, le poids d'état, compris entre 0.1 et 5.0, et l'intervalle de changement de direction de la cible, ajustable de 0.5 à 5.0 secondes. Elle affiche également en temps réel les trajectoires du robot intercepteur en rouge et du robot cible en bleu, leurs positions actuelles matérialisées par des disques de couleur cohérente, leurs vecteurs vitesse sous forme de flèches, ainsi que les statistiques de suivi : temps écoulé, distance d'erreur et vitesse courante du robot intercepteur.

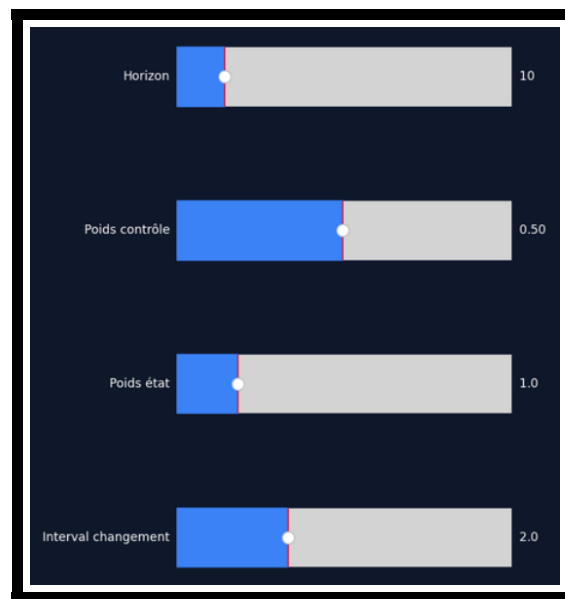


FIGURE 16 – Interface de simulation du contrôleur NMPC

Les deux graphiques ci-dessous [Figures 17 et 18] représentent chacun une interception d'une cible (bleue) par le robot intercepteur (rouge), avec leur trajectoire respective affichée. Ces scénarios sont générés aléatoirement comme expliqué précédemment. On observe, peu importe la simulation, que les déplacements de la cible soient doux ou brusques, le contrôleur permet à l'intercepteur de rejoindre sa cible.

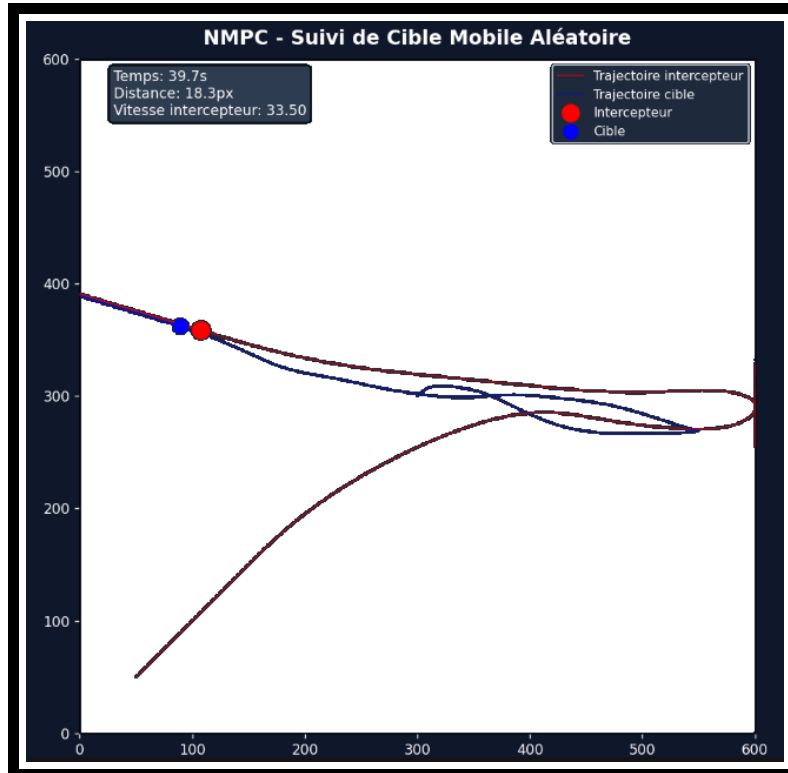


FIGURE 17 – Simulation d’interception avec un contrôleur NMPC

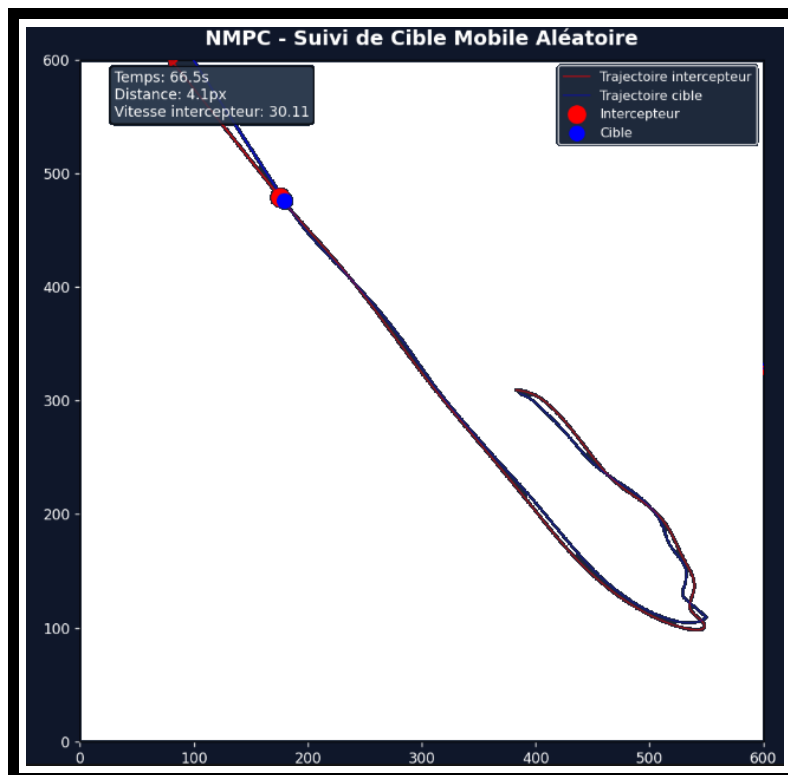


FIGURE 18 – Autre simulation d’interception avec un contrôleur NMPC

Pour mieux analyser les performances du contrôleur, deux graphes résumant les résultats de cinq simulations ont été tracés [Figure ??]. Celui du dessus représente

l'erreur de position entre l'intercepteur et sa cible. On constate que le contrôleur est suffisamment robuste pour offrir des performances très similaires dans les cinq cas aléatoires générés. Dans le graphique du dessous, c'est l'erreur d'orientation qui est représentée. En général, l'orientation reste très similaire. On observe cependant à $t = 12,5$ s environ un pic brusque pour la simulation 3 (courbe verte). Cela illustre que face à un changement brusque d'orientation de la cible (instantané dans cette simulation), l'intercepteur est incapable de suivre instantanément la trajectoire de la cible et doit réajuster peu à peu sa séquence de choix d'accélération. Un plus vaste choix d'accélération serait plus coûteux en calcul mais devrait permettre au NMPC de s'adapter plus rapidement à un brusque changement de trajectoire de ce type. Quoiqu'il en soit, l'erreur diminue relativement rapidement par la suite. On ajoutera enfin qu'il est normal que l'orientation entre les deux robots ne soient pas la même, l'objectif de l'intercepteur étant de couper le chemin pour rattraper sa cible.

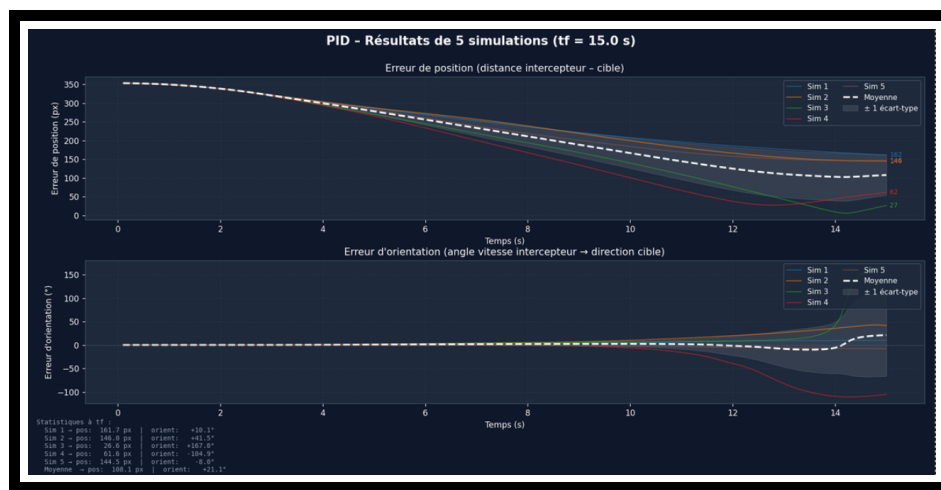


FIGURE 19 – Erreur en position et en orientation pour cinq simulations d'interceptions réalisées avec le contrôleur PID

À titre de comparaison, voici les résultats obtenus avec le contrôleur PID implémenté plus haut. On rappelle toutefois cependant qu'il n'est pas réellement possible de comparer deux contrôleurs dans la mesure où leurs performances dépendent en grande partie de leur réglage. Cependant, de par son fonctionnement interne le NMPC se démarque de son concurrent. On remarque facilement sur la courbe représentant l'erreur de position entre l'intercepteur et la cible pour cinq simulations du contrôleur PID que cette dernière est plus conséquente au bout de 15 s que pour le contrôleur NMPC, et que les résultats sont plus variables selon les simulations. Le PID se révèle donc moins précis et moins robuste pour satisfaire notre problématique d'interception.

Le NMPC présente plusieurs atouts majeurs pour notre application. Il permet au robot traqueur d'optimiser son comportement en temps réel, de trouver le meilleur compromis entre précision de suivi et économie d'énergie, et offre une bonne flexibilité grâce à ses paramètres réglables et à la possibilité d'y intégrer des contraintes physiques. Plusieurs axes d'amélioration ont néanmoins été identifiés. La recherche par force brute sur grille discrète, si elle illustre clairement les concepts fondamentaux du contrôle prédictif, reste peu efficace comparée aux solveurs d'optimisation

utilisés dans les NMPC industriels, qui explorent l'espace de contrôle de manière bien plus performante et large. La prédiction linéaire de la trajectoire cible constitue également une simplification forte : l'intégration de filtres de Kalman ou de réseaux de neurones permettrait d'anticiper bien plus précisément les mouvements de la cible. Enfin, l'intégration explicite des contraintes physiques réelles (limites d'accélération, présence d'obstacles dans l'environnement) renforcerait la robustesse du contrôleur en conditions opérationnelles. Trois soucis majeurs ont par contre été relevés. Le coût calculatoire constitue la contrainte principale, limitant en pratique l'horizon exploitable. La qualité du suivi dépend fortement de la prévisibilité de la cible : dans notre cas, les changements aléatoires de direction rendent la prédiction linéaire fréquemment erronée, ce qui dégrade les performances. Enfin, le réglage manuel des poids représente un défi opérationnel non négligeable, nécessitant un travail d'expérimentation important pour identifier le bon équilibre entre précision et fluidité, ce qui peut s'avérer coûteux en temps lors du déploiement sur un système réel.

II.3 Guidage par asservissement visuel

Afin de fournir aux algorithmes de contrôle des données sur la position de la cible par rapport à l'intercepteur pour qu'ils puissent calculer la consigne à envoyer aux moteurs, un algorithme d'asservissement visuel a été mis en place.

L'asservissement visuel regroupe les méthodes qui ferment une boucle de commande à partir de mesures visuelles. Il en existe trois grandes familles, et le choix entre elles dépend des contraintes du projet.

Algorithmes de type IBVS 2D

Pour les algorithmes Image-Based Visual Servoing 2D (IBVS 2D), l'erreur est définie entièrement dans l'espace image :

$$\mathbf{e} = \mathbf{s} - \mathbf{s}^*, \quad \mathbf{s} \in \mathbb{R}^n$$

où $\mathbf{s}$ regroupe des primitives 2D comme des coordonnées de points ou des moments d'image. La loi de commande utilise une matrice d'interaction J_{2D} :

$$\dot{\mathbf{s}} = J_{2D} \mathbf{v}, \quad \mathbf{v} = -\lambda J_{2D}^{-1} \mathbf{e}$$

Cette approche est simple à mettre en œuvre et ne nécessite pas de reconstruction 3D. Son principal défaut est que J_{2D} dépend de la profondeur Z , mais cette dépendance est ignorée ou approximée. Cette approximation fonctionne bien quand la distance à la cible varie peu, mais devient source d'imprécision lors d'une phase d'approche sur plusieurs dizaines de mètres comme dans notre cas.

Algorithmes de type PBVS 3D

Les algorithmes de type PBVS (Position-Based Visual Servoing) reconstruisent la pose complète de la cible en définissant l'erreur en 3 dimensions :

$$\mathbf{e} = \begin{bmatrix} \mathbf{t} - \mathbf{t}^* \\ \boldsymbol{\theta} \mathbf{u} \end{bmatrix}$$

Les trajectoires générées sont propres et la convergence peut être garantie globalement. En revanche, cette approche nécessite une estimation de pose précise, ce qui implique des marqueurs, une stéréovision ou une reconstruction dense. En milieu marin, avec les réflexions sur l'eau et les embruns, cette hypothèse n'est pas réaliste. Nous avons donc écarté cette approche.

Algorithmes d'asservissement 2,5D

L'approche 2,5D combine les deux familles précédentes. Elle utilise des primitives d'image comme les algorithmes IBVS, mais enrichit la matrice d'interaction avec une estimation partielle de la profondeur. L'erreur est définie dans un espace hybride :

$$\mathbf{e} = \begin{bmatrix} \text{sim}g - \text{sim}g^* \\ \log\left(\frac{Z^*}{Z}\right) \end{bmatrix}$$

Seul le ratio de profondeur Z^*/Z est nécessaire, et non la profondeur absolue. Cela suffit à construire une matrice d'interaction exacte, ce qui améliore le comportement lors des phases d'approche sans nécessiter la reconstruction de pose complète du PBVS.

Approche retenue : 2,5D adapté à la détection par boîte

Notre implémentation s'inscrit dans le cadre du 2,5D. Les primitives $(x, \log h)$ sont des grandeurs image, mais la matrice J est construite avec une estimation de Z tirée de la hauteur apparente du bounding box :

$$\hat{Z} = \frac{f_y H}{h}$$

où H est la hauteur réelle connue de la cible. Par rapport à un algorithme IBVS 2D pur qui utiliserait une valeur de Z constante, notre approche met à jour J à chaque instant en fonction de la distance courante. Ce choix est possible grâce à YOLO, qui fournit directement la hauteur h du bounding box.

Ce choix est justifié par trois raisons. D'abord, la reconstruction de pose complète est difficile en milieu marin sans marqueurs ni stéréovision. Ensuite, la hauteur apparente h est une information de profondeur robuste, directement disponible depuis le bounding box. Enfin, $\hat{Z}$ est suffisant pour obtenir une matrice J bien conditionnée sur toute la plage de distance opérationnelle.

Hypothèses de travail

Le modèle repose sur quatre hypothèses. La cible est supposée approximativement verticale et sa hauteur réelle H est connue. Le mouvement de la plateforme est supposé plan, ce qui signifie que le tangage et le roulis sont négligeables. La caméra est fixe, frontale et calibrée. Enfin, le bounding box est supposé suffisamment stable pour fournir des primitives utilisables, ce qui justifie l'utilisation du filtrage.

Modèle de la caméra

On utilise le modèle projectif standard :

$$u = f_x \frac{X}{Z} + c_x, \quad v = f_y \frac{Y}{Z} + c_y \quad (2)$$

où (X, Y, Z) sont les coordonnées 3D de la cible dans le repère caméra, (u, v) les coordonnées pixel, et (f_x, f_y, c_x, c_y) les paramètres intrinsèques.

Primitives d'images et estimation de profondeur

On définit la primitive de centrage normalisé :

$$x = \frac{u - c_x}{f_x} \quad (3)$$

La seconde primitive est la hauteur h du bounding box en pixels. Pour une cible de hauteur réelle H , la géométrie projective donne :

$$h \approx \frac{f_y H}{Z} \implies \frac{1}{Z} \approx \frac{h}{f_y H} \quad (4)$$

Cette relation est valide sous l'hypothèse d'une cible verticale et d'un faible angle de tangage.

Fonction d'erreur

L'objectif est de centrer la cible horizontalement et d'atteindre la distance désirée. L'erreur est :

$$e_1 = x, \quad e_2 = \log h - \log h_{\text{des}} \quad (5)$$

Le terme $\log h$ est préféré à h car il linéarise la dépendance en distance. Puisque $\log h = \log(f_y H) - \log Z$, l'erreur e_2 est proportionnelle à l'écart de profondeur en échelle logarithmique. En pratique, cela rend la commande moins agressive à grande distance et plus réactive à courte distance, ce qui est un comportement adapté à une phase d'interception.

Cinématique image

Le robot dispose de deux degrés de liberté : la vitesse linéaire avant u_c et la vitesse de lacet ω . La dérivée temporelle des primitives est :

$$\dot{x} = \frac{x}{Z} u_c - (1 + x^2) \omega \quad (6)$$

$$\log \dot{h} = \frac{1}{Z} u_c - x \omega \quad (7)$$

Matrice d'interaction

En substituant l'estimation de profondeur (4) dans (6)–(7), on obtient :

$$J = \begin{bmatrix} \frac{h x}{f_y H} & -(1 + x^2) \\ \frac{h}{f_y H} & -x \end{bmatrix} \quad (8)$$

C'est la matrice implémentée dans `visual_control.cpp`. Son déterminant vaut $\det(J) = -h/(f_y H)$, qui s'annule uniquement si $h = 0$, c'est-à-dire si la cible quitte complètement le champ de vision. Cette singularité est gérée par le mécanisme de timeout décrit dans la sous section II.3.

Loi de commande

La loi de commande est :

$$\begin{bmatrix} u_c \\ \omega \end{bmatrix} = -\lambda J^{-1} \mathbf{e} \quad (9)$$

avec $\lambda > 0$. La commande est saturée par $\pm V_{\max}$ avant d'être envoyée aux actionneurs, et annulée si $|\det(J)| < 10^{-6}$.

Détection par YOLO

YOLO (*You Only Look Once*) est le détecteur utilisé en conditions réelles. Il prédit en un seul passage réseau les boîtes englobantes et les scores de confiance, ce qui lui donne une latence faible, compatible avec une boucle de contrôle à 20 Hz. Sa sortie est formatée en message ROS2 `vision_msgs/Detection2DArray`, directement utilisable par le contrôleur.

Le modèle utilisé est Odessa, entraîné sur des données maritimes pour la détection d'embarcations. Un modèle généraliste aurait eu du mal à distinguer un bateau des vagues ou du port. Odessa est déployé sur une NVIDIA Jetson via TensorRT, ce qui réduit la latence de détection à quelques millisecondes grâce à l'accélération GPU.

Dans le contexte de l’asservissement visuel, la précision des boîtes d’ancrage conditionne directement la qualité des primitives x et h . Une détection bruitée ou instable se traduit par des oscillations de commande, d’où l’importance du filtrage décrit en section II.3.

Suivi par ByteTrack

L’algorithme de propagation de détection ByteTrack a été intégré après les essais en mer pour améliorer la robustesse du pipeline face à deux problèmes identifiés. Le premier est l’instabilité des détections réalisées par YOLO entre deux images : la position de la boîte d’ancrage varie légèrement d’une image à l’autre même si la cible est fixe, ce qui génère des micro-oscillations de commande. Le second est la gestion des cibles multiples : sans suivi, le contrôleur peut changer de cible entre deux frames quand plusieurs bateaux sont présents dans l’image.

L’algorithme ByteTrack résout ces deux problèmes en associant temporellement les détections par filtre de Kalman. Contrairement aux trackers classiques qui ignorent les détections de faible confiance, ByteTrack les utilise également, ce qui limite les pertes de piste lors d’occultations partielles. Chaque piste reçoit un identifiant stable et les dimensions des boîtes d’ancrage sont lissées temporellement.

ByteTrack a été intégré à nos algorithmes à la suite des expérimentations sur l’intercepteur bateau. Son impact sur les performances reste à quantifier lors de futurs essais.

Détection couleur avec OpenCV

Pour les phases de test, une détection par segmentation couleur HSV est disponible en remplacement de YOLO. Elle identifie les pixels d’une couleur cible (jaune pour un cône de chantier, par exemple), extrait la bounding box du plus grand contour, et publie le résultat sur le même topic `detections_output`. Le contrôleur ne fait aucune différence entre les deux sources.

Cette approche ne nécessite pas de GPU ni de modèle entraîné, et s’exécute sur tout ordinateur standard. Elle a permis de valider la loi de commande sur rover bien avant que l’intégration YOLO/Jetson soit disponible, ce qui a considérablement accéléré le développement.

Architecture ROS 2

Le contrôleur est le nœud C++ `ControlNode` (`visual_control.cpp`). Il s’abonne à `detections_output` (OU `tracked_detections` si ByteTrack est actif) et publie sur `/mavros/setpoint_velocity/cmd_vel_unstamped`.

Deux callbacks sont séparés. Le premier, `control_callback`, est déclenché à chaque détection reçue : il extrait et filtre les primitives x et h . Le second, `control_timer_callback`, s'exécute à fréquence fixe (20 Hz) et calcule la commande à partir des primitives filtrées. Cette séparation découple la fréquence de détection — variable selon la charge de YOLO — de la fréquence de contrôle, qui reste constante. Sans cela, une variation du framerate de YOLO se répercuterait directement sur la régularité de la commande.

Filtrage exponentiel des primitives

Pour réduire le bruit des détections YOLO, un filtre exponentiel du premier ordre est appliqué à x et h :

$$x_f \leftarrow \alpha x_{\text{mes}} + (1 - \alpha) x_f, \quad h_f \leftarrow \alpha h_{\text{mes}} + (1 - \alpha) h_f \quad (10)$$

Le paramètre $\alpha = \text{filter_alpha}$ contrôle le compromis entre lissage et réactivité. La valeur $\alpha = 0.4$ a été choisie empiriquement. Elle est suffisamment basse pour lisser le jitter, et suffisamment haute pour que le filtre suive correctement une cible en mouvement. Ce réglage s'est révélé particulièrement utile lors des essais avec la caméra noir et blanc, où les détections étaient plus bruitées que dans les conditions nominales.

Mesures de sécurité

Trois mécanismes de sécurité ont été mis en place. Si aucune détection n'est reçue depuis plus de `timeout_sec` secondes, la commande est mise à zéro et le bateau s'arrête. Si le déterminant de J passe sous 10^{-6} , la commande est aussi annulée pour éviter une instabilité numérique. Enfin, u_c et ω sont bornés par $\pm V_{\text{max}}$ via `std::clamp` pour protéger les actionneurs.

Ces mécanismes ont joué un rôle concret lors des essais en conditions dégradées : le filtrage et le timeout ont permis au système de maintenir un comportement stable malgré des détections intermittentes sur caméra noir et blanc.

Comportement en boucle fermée

Le comportement du contrôleur peut être résumé simplement. Quand la cible est décalée sur le côté ($e_1 \neq 0$), le lacet ω la recentre. Quand la cible est trop loin ($e_2 > 0$), la commande avant u_c accélère. Quand on est trop près ($e_2 < 0$), u_c ralentit ou s'inverse. Quand les deux erreurs sont nulles, les deux commandes s'annulent et le bateau maintient sa position relative par rapport à la cible.

Paramètres et réglage

Les paramètres du contrôleur sont listés dans le tableau suivant avec leur valeur par défaut et leur rôle.

Paramètre	Symbole	Valeur par défaut	Rôle
f_x, f_y	f_x, f_y	600.0 px (pixels)	Focale caméra
c_x, c_y	c_x, c_y	320, 240	Centre optique
height_target	H	0.63 m	Hauteur réelle de la cible
desired_height	h_{des}	150 px	Distance d'interception visée
lamda	λ	0.5	Gain de commande
Vmax	V_{max}	10.0 m/s	Saturation des commandes
filter_alpha	α	0.4	Lissage des détections
timeout_sec	—	5.0 s	Délai avant arrêt d'urgence
control_rate_hz	—	20.0 Hz	Fréquence de contrôle

Le gain λ est le paramètre le plus important. Un λ élevé accélère la convergence mais peut provoquer des oscillations. Un λ faible donne un comportement plus lent mais plus stable. La valeur $\lambda = 0.5$ a été trouvée sur le rover par essais successifs et a été réutilisée sans modification sur le bateau, avec de bons résultats.

Le paramètre h_{des} fixe la distance finale d'interception. Une grande valeur correspond à une cible qui occupe une grande partie de l'image, donc à une distance finale faible.

Analyse des limites de l'asservissement visuel

La première limite concerne l'estimation de profondeur. Puisque $\hat{Z} = f_y H / h$, une erreur sur H se propage directement dans la commande. Une erreur de 10% sur H donne une erreur de 10% sur u_c . En pratique, H est fixé à une valeur représentative et ne change pas en ligne, ce qui est acceptable pour un bateau dont la hauteur est approximativement connue.

La calibration caméra est également critique. Une erreur sur f_y fausse l'estimation de $1/Z$, et une erreur sur c_x décale l'erreur de centrage e_1 . Il est important de calibrer dans les conditions réelles d'utilisation.

L'hypothèse de mouvement plan est la plus contraignante en milieu marin. Par forte houle, le tangage et le roulis introduisent un biais sur h et x que le contrôleur ne compense pas.

Sans ByteTrack, le contrôleur suit la première détection retournée par YOLO. En présence de plusieurs bateaux, cela peut conduire à un suivi la mauvaise cible.

Enfin, en cas de perte de détection prolongée, le système s'arrête mais n'effectue aucun comportement de recherche pour retrouver la cible.

Simulations

En dehors des deux semaines d'expérimentations à Guerlédan, nous n'avons pas accès aux USV IM1800. Le rover n'étant pas suffisant pour valider avec l'intégralité de nos algorithmes de contrôle, notre seule option pour ce faire a été de créer des simulations. Nous avons fait le choix de créer deux simulations sur deux simulateurs différents car chacun d'eux est spécialisé dans un domaine.

Le premier, Gazebo [Figure 20], est reconnu et largement utilisé dans la communauté robotique. Il est en effet pensé pour s'intégrer avec ROS2, est très fidèle d'un point de vue simulation physique et dispose de nombreuses librairies ou modules prêts à l'emploi. Il est par exemple très simple d'ajouter des capteurs classiques (lidar, caméra, GNSS. . .) à notre simulation et il existe des librairies spécialement conçues pour les simulations marines. Cependant ce logiciel est complexe à prendre en main et ne permet pas un rendu graphique très réaliste. Par conséquent Gazebo est principalement utilisé pour tester des algorithmes de contrôle complexes.



FIGURE 20 – Logo du logiciel de simulation Gazebo



FIGURE 21 – Logo du moteur de jeux vidéos Unity

Le deuxième, Unity [Figure 21] est à l'origine un moteur de jeux vidéo mais est de plus en plus utilisé pour en robotique pour ses capacités de simulation. Contrairement à Gazebo, il permet une grande fidélité visuelle et esthétique. Bien que moins précis dans la simulation physique, il est bien plus simple à prendre en main et sa communauté est très active. Ce logiciel est donc principalement utilisé pour des démonstrations clients ou pour tester des algorithmes de vision.

Il est aussi important de noter que chaque roboticien a ses préférences en termes de simulateur, et qu'il est donc pratique d'avoir deux simulateurs différents en fonction des cas d'utilisation et des connaissances/préférences des futurs utilisateurs.

Simulation réalisée sur Gazebo

La première simulation a été réalisée sur Gazebo Harmonic. Elle inclut 2 USV (un intercepteur et une cible), eux même composés de plusieurs capteurs et actionneurs. L'USV intercepteur simulé embarque donc 3 caméras (une monochrome, une RGB et une thermique), un récepteur GNSS, une IMU, un lidar et 2 moteurs. Le radar ne fait pas partie des capteurs disponibles par défaut dans Gazebo mais il serait assez aisé d'en simuler un si nécessaire. Tous les capteurs pouvant l'être ont été bruités et tous les messages Gazebo des capteurs et des moteurs ont été traduits en messages ROS2 [Figure 22].

```
→ interceptor_ws ros2 topic list
/clock
/interceptor_sim/interceptor/cam_mono/image
/interceptor_sim/interceptor/cam_rgb/image
/interceptor_sim/interceptor/engine_left/ang_vel
/interceptor_sim/interceptor/engine_right/ang_vel
/interceptor_sim/interceptor/gps
/interceptor_sim/interceptor/imu
/interceptor_sim/interceptor/lidar
/interceptor_sim/interceptor/lidar_points
/interceptor_sim/target/cam_mono/image
/interceptor_sim/target/cam_rgb/image
/interceptor_sim/target/engine_left/ang_vel
/interceptor_sim/target/engine_right/ang_vel
/interceptor_sim/target/gps
/interceptor_sim/target/imu
/interceptor_sim/target/lidar
/interceptor_sim/target/lidar_points
```

FIGURE 22 – Messages ROS2 traduits à partir des messages Gazebo

La simulation de l'environnement est faite avec l'aide du package `asv_wave_sim`. Ce dernier simule les vagues de manière réaliste, que ce soit physiquement ou visuellement. De plus, il prend en charge la simulation dynamique de l'USV en s'inspirant du modèle de Fossen. N'ayant pas les coefficients hydrodynamiques de l'IM1800, ces derniers ont été choisis empiriquement pour la simulation. La figure 23 donne une idée du rendu visuel obtenu.

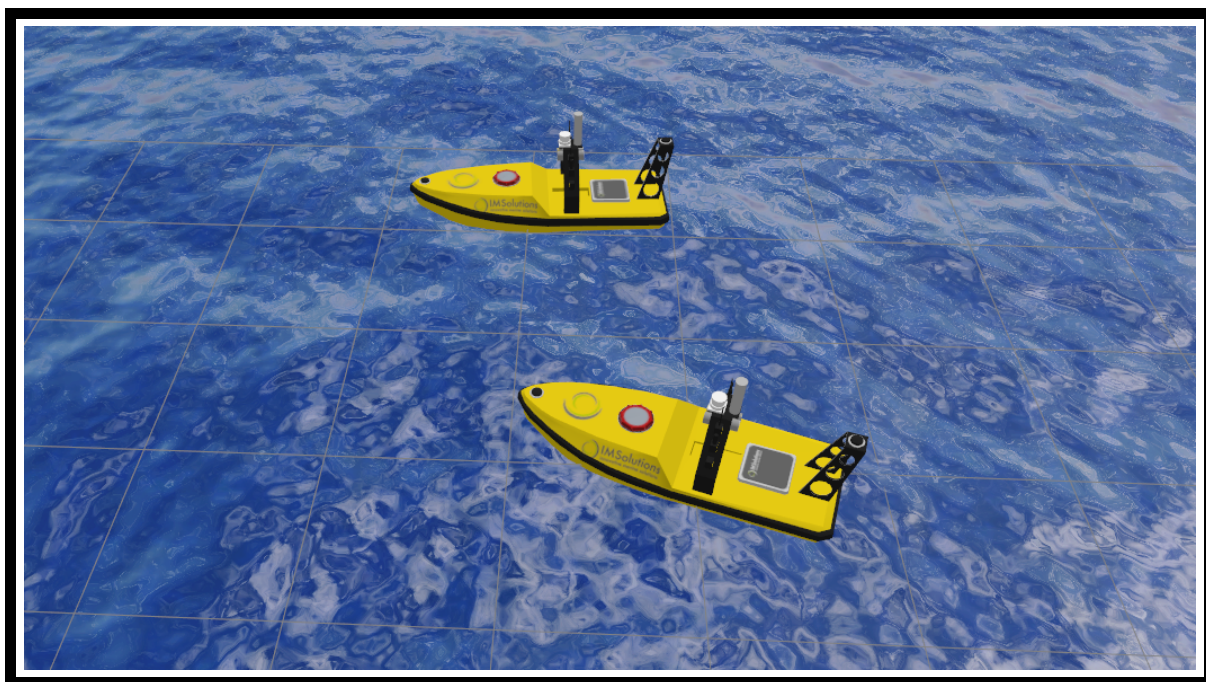


FIGURE 23 – Rendu visuel de la simulation Gazebo

Un contrôle simple basé sur un contrôleur PID et sur une reconnaissance d'objet par intelligence artificielle a été mis en place pour tester la simulation. Un modèle YOLO reconnaît le bateau cible sur l'image de la caméra de l'intercepteur, le PID se charge de centrer le bateau cible sur l'image tout en ajoutant une vitesse d'avance non nulle. Si la boîte de délimitation de l'USV cible dépasse une certaine aire (ce qui équivaut à une cible proche), on arrête les moteurs. Si l'intercepteur ne reconnaît aucun bateau, il tourne doucement sur lui-même jusqu'à ce que YOLO en identifie un.



FIGURE 24 – Cible identifié par l'intercepteur grâce à YOLO sur Gazebo

Malgré le fait que le modèle YOLO utilisé ne soit pas entraîné à reconnaître spécifiquement notre bateau et qu'il reste très léger afin de pouvoir tourner sur nos ordinateurs personnels, on obtient néanmoins de très bons résultats avec le contrôleur PID implémenté.

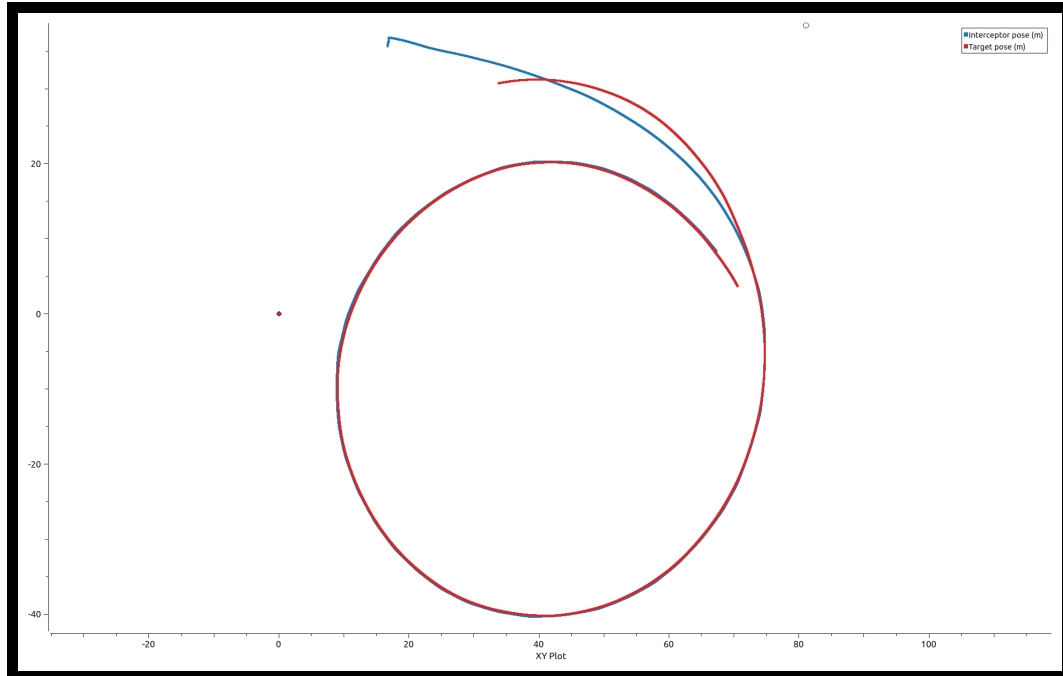


FIGURE 25 – Trajectoire de la cible et de l'intercepteur, on constate que l'intercepteur réussit à suivre sa cible

Pour conclure sur la partie traitant de la simulation avec Gazebo, nous avons donc réussi à créer une simulation fonctionnelle, réaliste et facilement exploitable par les potentiels futurs utilisateurs.

Simulation du contrôleur NMPC avec Unity

Afin de tester et valider les algorithmes de contrôle et de guidage développés au cours du projet, un nouvel environnement de simulation a été implémenté sous Unity 6, en complément de la simulation initialement réalisée sous Gazebo, conformément aux recommandations des encadrants. Ce changement d'outil vise à bénéficier d'un moteur graphique et physique plus flexible et jeune, permettant de concilier réalisme dynamique et visuel au sein d'un même environnement. D'une part, la simulation doit reproduire avec fidélité le comportement dynamique de l'intercepteur réel, en ce qui concerne des effets hydrodynamiques simplifiés, les interactions avec la surface de l'eau et la réponse aux commandes de propulsion. Cette fidélité est essentielle pour pouvoir valider les algorithmes de contrôle, guidage et navigation développés. D'autre part, l'environnement doit présenter un niveau de réalisme visuel suffisant pour permettre l'entraînement et l'évaluation de modèles de reconnaissance visuelle, comme YOLO. Dans ce contexte, la qualité du rendu, la gestion

de l'éclairage, des ombres et des textures jouent un rôle déterminant dans la pertinence des données synthétiques générées.

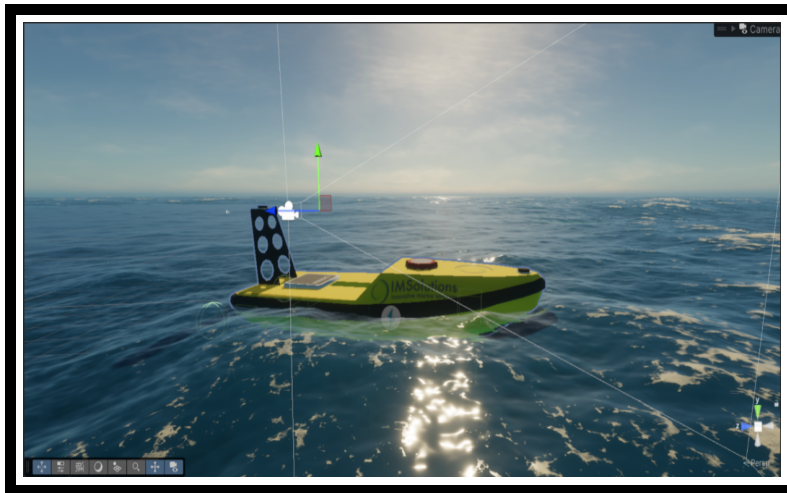


FIGURE 26 – Vue de l'intercepteur sur Unity (notez le réalisme avec lequel sont rendus la mer et l'éclairage de la scène)

Pour répondre à ces exigences, la simulation a été développée à l'aide du High Definition Render Pipeline (HDRP) de Unity. Le HDRP est un pipeline de rendu destiné aux plateformes performantes et conçu pour produire des images à haut niveau de réalisme physique. Il repose sur un modèle d'éclairage physiquement basé qui simule de manière cohérente les interactions entre la lumière et les matériaux (réflexion, réfraction, rugosité, ...). Dans le cadre de la simulation maritime, les fonctionnalités du HDRP sont particulièrement utiles pour représenter de manière crédible la surface de l'eau, les variations d'éclairage en fonction de la position du soleil, et les reflets du bateau sur l'environnement. La figure 26 illustre le rendu visuel de la simulation de l'intercepteur à l'aide du HDRP.

L'utilisation du HDRP étant plus coûteuse en calcul, nous avons optimisé la configuration graphique en désactivant les effets visuels non essentiels (brouillard volumétrique, post-traitement avancé, particules) pour permettre à la simulation de tourner sur de plus petites configurations. Ainsi, la simulation développée sous Unity 6 constitue un environnement cohérent et modulable, adapté à la fois à l'évaluation des performances des lois de commande et au développement de modules de perception embarquée, tout en restant compatible avec des contraintes de calcul réalistes.

Pour simuler dynamiquement l'eau et ses forces, on utilise la base fournie par le HDRP et on ajoute un code de flottaison implémentant un modèle simplifié de flottabilité appliqué à un corps rigide (rigidbody), en interaction avec une surface d'eau issue du HDRP. À chaque itération de physique, une fraction de la gravité est d'abord appliquée au point courant, ce qui permet, lorsqu'il est dupliqué sur plusieurs points flottants, de répartir les forces et de stabiliser l'objet (par exemple une coque de bateau). Le script évalue ensuite la surface qui projette la position du flotteur sur la surface d'eau et retourne la hauteur locale de l'eau. Si le point considéré se situe sous cette surface, une force de poussée d'Archimède simplifiée est calculée : son intensité dépend de la profondeur d'immersion relative et est pondérée par un coef-

ficient de déplacement. Cette force verticale, proportionnelle à la gravité, est appliquée au point considéré afin de générer à la fois une sustentation et un moment de redressement. Enfin, des termes d'amortissement linéaire et angulaire sont ajoutés, proportionnels à la vitesse et à la vitesse de rotation du corps rigide, afin de modéliser la traînée hydrodynamique et de stabiliser le mouvement dans l'eau. L'ensemble constitue un modèle physique discret, peu coûteux en calcul, permettant de simuler de manière réaliste le comportement flottant d'un véhicule de surface. A noter qu'un script (*WaterExcluder.cs*) a été récupéré d'un template Unity officiel pour améliorer l'interaction entre l'eau et le maillage (mesh) et éviter des recouvrements visuels.

Des modèles simplifiés de bateaux ont été intégrés dans la simulation afin de valider les principes de dynamique et de propulsion. Ces modèles reposent sur une architecture à propulsion différentielle, composée de deux moteurs indépendants positionnés à l'arrière, respectivement à bâbord et à tribord. Les forces de poussée associées à chaque moteur ont été implémentées. Un équilibrage des commandes résulte en une trajectoire rectiligne. Pour virer d'un côté, on active le moteur du côté opposé et on désactive celui du même côté. Le script *Thruster_force.cs* implémente ce contrôleur de propulsion différentielle en associant deux propulseurs, chacun représenté par un objet Transform stockant la pose de l'objet considéré, auxquels sont appliquées des forces. Les touches directionnelles pilotent directement les niveaux de poussée : la flèche haut active les deux moteurs pour avancer, tandis que les flèches gauche et droite créent un différentiel de poussée produisant une rotation sur place ou un virage serré. Le script configure également les paramètres de traînée linéaire et angulaire du corps rigide, ajustables en temps réel depuis l'inspecteur, afin de simuler le glissement et la résistance à la rotation dans l'eau.



FIGURE 27 – L'intercepteur vu de haut. Les sphères blanches à l'arrière de l'USV représentent ses moteurs et les cubes blancs, ses flotteurs

Sur la figure 27, on peut observer le M1800 Interceptor, avec ses deux moteurs arrières représentés par des sphères blanches et ses quatre frotteurs représentés par des cube blancs. Une caméra virtuelle a aussi été installée dessus pour permettre des observations similaires à celle de l'USV réel. Le robot cible a été conçu à l'identique, à ceci près que ses moteurs sont bridés pour diminuer le temps d'interception et donc de simulation. D'autres capteurs ont été ajoutés au modèle comme un GPS, ou un radar. Ces capteurs sont simulés par des codes simples.

Pour passer à une implémentation des contrôleurs PID et NMPC, il a tout d'abord fallu ajouter une cible mobile comme celle vue dans la partie II. Un script, *Random-Target.cs* a donc été créé pour simuler la cible de la même façon que précédemment attaché à un second bateau.

Le premier algorithme à avoir été testé est le PID afin de valider ou non le comportement de la simulation. Une fois la simulation fonctionnelle, le contrôleur NMPC a lui aussi été testé via le script *NMPCTracker.cs* qui implémente le contrôleur et commande directement les deux moteurs de l'USV intercepteur.



FIGURE 28 – Image générée par la caméra virtuelle placée sur l'intercepteur

L'image ci-dessus [28] est fournie par la caméra placée directement sur l'USV intercepteur simulé. Sa qualité est suffisante pour utiliser des modèles d'intelligence artificielle YOLO de la même manière que sur le terrain.

Plusieurs tests d'interception ont été menés avec le contrôleur NMPC dans Unity. Pour chacun d'eux, nous avons obtenu un résultat correspondant à nos attentes. Les images ci-dessous [Figures 29, 30, 31 et 32] ont été prises à différents instants de la même simulation d'interception. On peut remarquer que l'intercepteur rattrape petit à petit la cible et finit par l'intercepter.

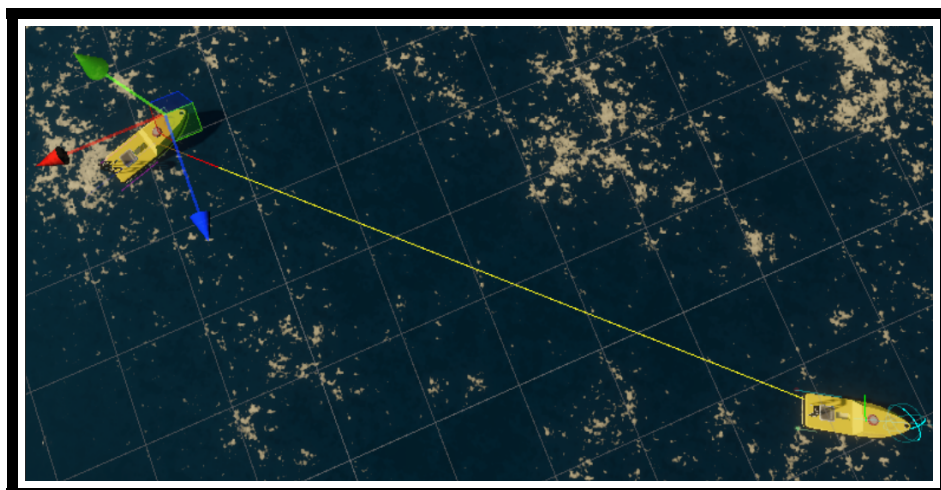


FIGURE 29 – Position de l'intercepteur et de la cible à l'instant initial ($t = t_0$)

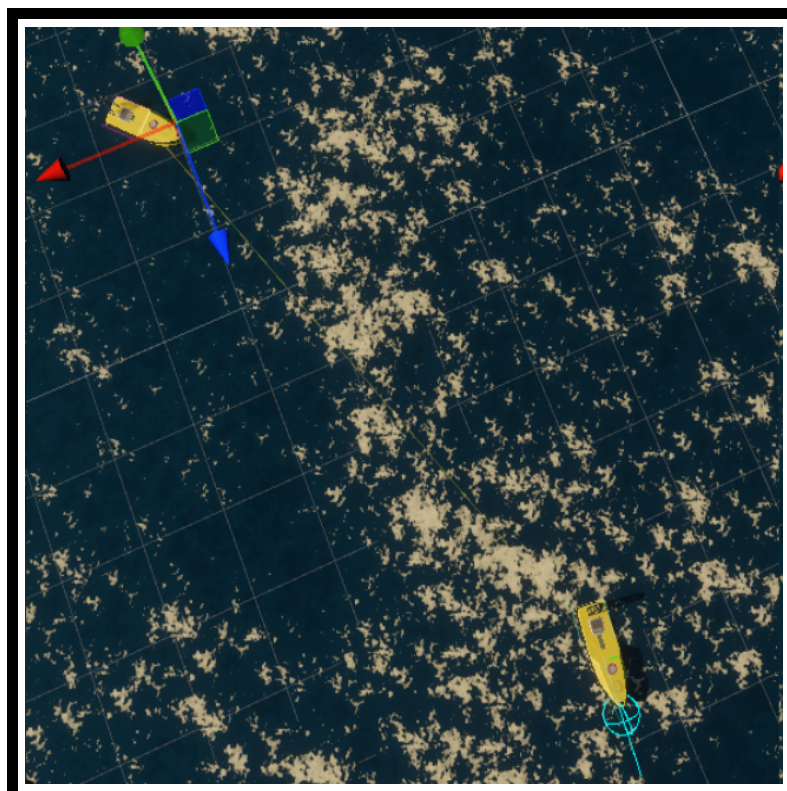


FIGURE 30 – Position de l'intercepteur et de la cible à l'instant $t = t_1 > t_0$

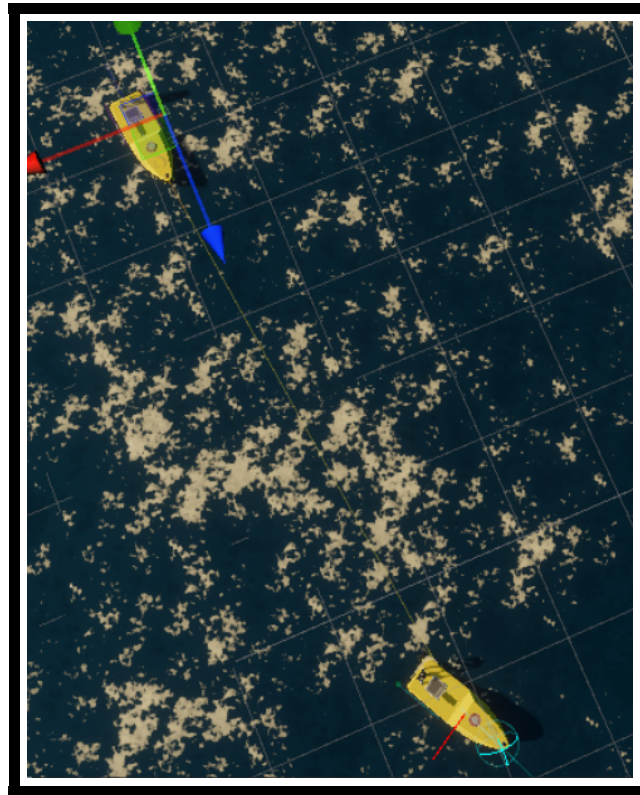


FIGURE 31 – Position de l'intercepteur et de la cible à l'instant $t = t_2 > t_1$

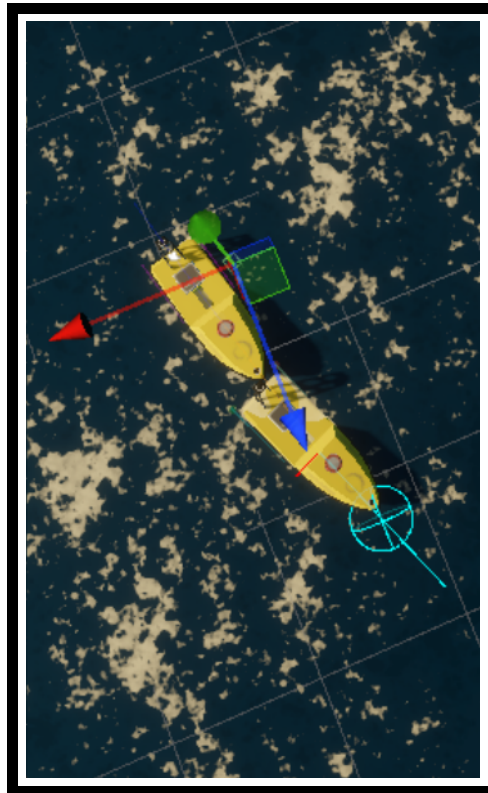


FIGURE 32 – Position de l'intercepteur et de la cible à l'instant final. L'intercepteur a rattrapé la cible.

Pour une visualisation plus simple et plus pertinente scientifiquement, un code *TrajectoryLogger.cs* a été implémenté dans Unity pour récupérer les coordonnées des deux USV et les écrire dans un fichier. Elles ont ensuite été affichées grâce à Python [Figure 33]. Les points correspondent à la position initiale des bateaux et les étoiles à leur position finale. La trajectoire rouge est associée à la cible tandis que la bleue est associée à l'intercepteur. En observant ces trajectoires, on remarque que l'intercepteur a très vite rejoint la cible avant de s'adapter à son rythme et à ses déplacements. Il semble maintenant impossible à la cible de lui échapper. Malgré les contraintes dynamiques de l'eau et des USVs, on obtient un résultat similaire à ceux obtenus dans la partie II avec le contrôleur NMPC.

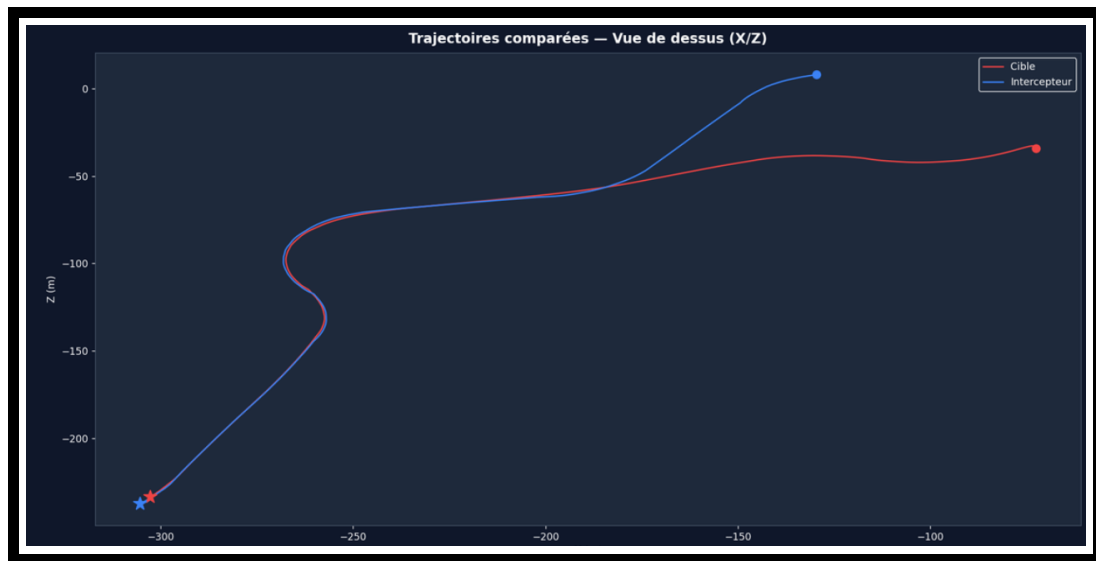


FIGURE 33 – Position de l'intercepteur et de la cible tout au long d'une simulation avec Unity. Il est à noter que l'interception a été réalisée avec succès.

III Présentation de l'interface homme-machine (IHM)

Parmi les attentes du projet figurait la création d'une interface homme machine (IHM) adaptée aux missions à réaliser avec l'intercepteur. Nous avons donc conçu l'IHM demandée avec pour objectifs :

- d'afficher les différents paramètres de l'intercepteur et du rover (vitesse, position, vue depuis les caméras, etc...) pour que leurs opérateurs puissent agir en conséquence si un événement imprévu advient.
- de pouvoir armer et désarmer le drone, de même qu'interrompre la mission si nécessaire grâce à un bouton d'arrêt d'urgence.
- de pouvoir lancer les mission à distance en étant connecté en Secure Shell (SSH) avec le rover ou l'intercepteur.

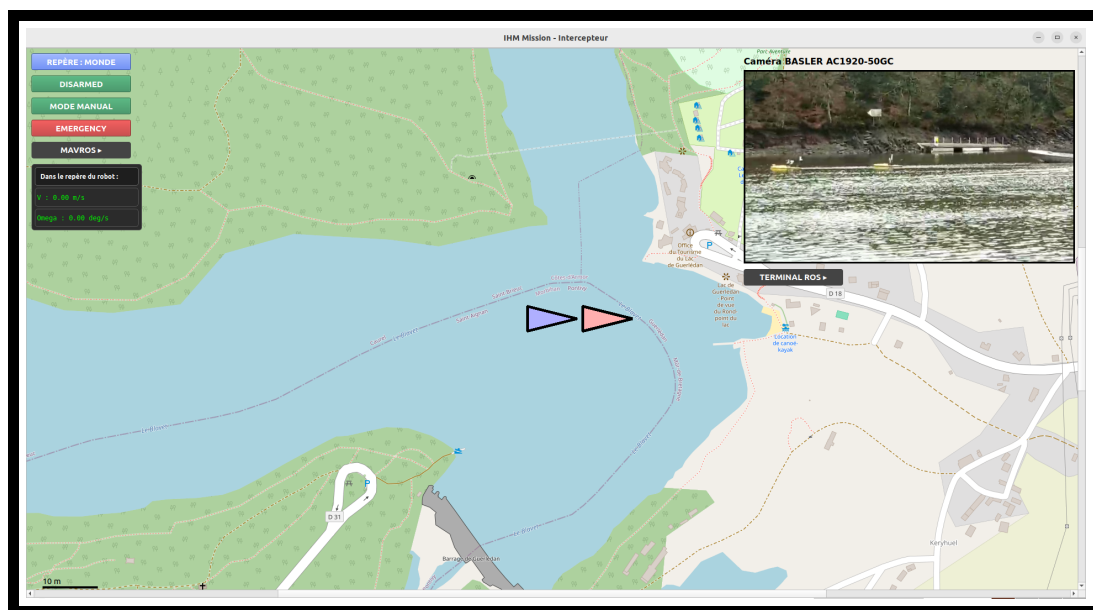


FIGURE 34 – Vue générale de l'IHM

L'IHM est composé de deux programmes. Le premier est un nœud ROS2 chargé de récupérer les données utiles (position, orientation, mode de guidage et armement de l'intercepteur, etc...) et de les traiter (en changeant de référentiel les différentes positions par exemple) tandis que le deuxième se charge de l'affichage.

La première chose que l'IHM permet est une visualisation de la position et de l'orientation du drone intercepteur, ainsi que de celles de la cible. En admettant qu'elle soit disponible, il serait également aisé d'ajouter un affichage de la position de la cible estimée par l'algorithme YOLO tournant sur l'intercepteur. Ces positions sont affichées sur une carte générée à l'aide de Leaflet, une bibliothèque JavaScript utilisée pour réaliser des cartes interactives [Figure 35]. Il est à noter que cette approche nécessite d'être connecté à Internet pour pouvoir récupérer les tuiles de la carte dont on a besoin.

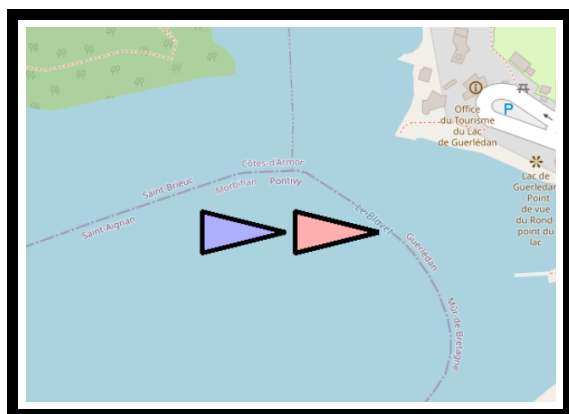


FIGURE 35 – Intercepteur (en bleu) et cible (en rouge) sur la carte générée grâce à Leaflet

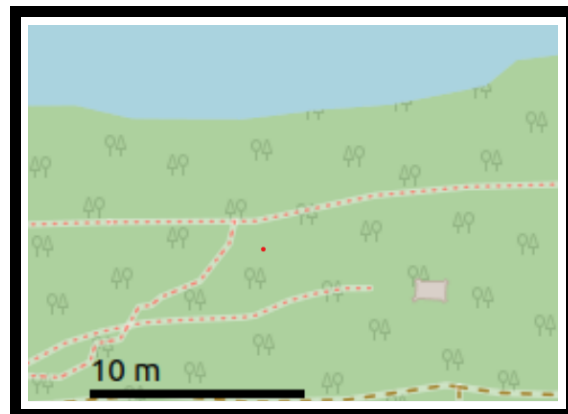


FIGURE 36 – Échelle présente en bas à gauche de la fenêtre de l'IHM

Dans le but d'améliorer le confort de l'opérateur, celui-ci a également la possibilité de zoomer et de translater la vue dans une certaine mesure. Une barre indiquant l'échelle est également présente en bas à gauche de la fenêtre [Figure 36]. Cette dernière s'adapte en fonction du facteur de zoom.

L'IHM permet également de visualiser les images reçues par l'une des caméras du drone, ainsi que les détections des cibles potentielles réalisées par YOLO sous forme de boîtes d'ancrages (bounding boxes). Ces images s'affichent en haut à droite de l'écran [Figure 34]. Il est à noter que les images traitées par YOLO sont privilégiées aux images non traitées lorsqu'elles sont disponibles.

En plus de fournir à l'opérateur des informations sur le drone et son environnement, l'IHM apporte également plusieurs fonctionnalités pour le décharger de commandes potentiellement longues à entrer dans un terminal. L'ensemble des boutons et menus déroulants permettant de paramétrer et de lancer les différentes commandes est présent en haut à gauche de la fenêtre de l'IHM [Figure 34]. On y trouve également un bouton pour changer le repère utilisé pour l'affichage (repère du monde ou repère du robot) [Figure 37]. L'implémentation de l'affichage dans le repère du robot n'a néanmoins pas été menée à bien.



FIGURE 37 – Panneau des commandes de l'IHM

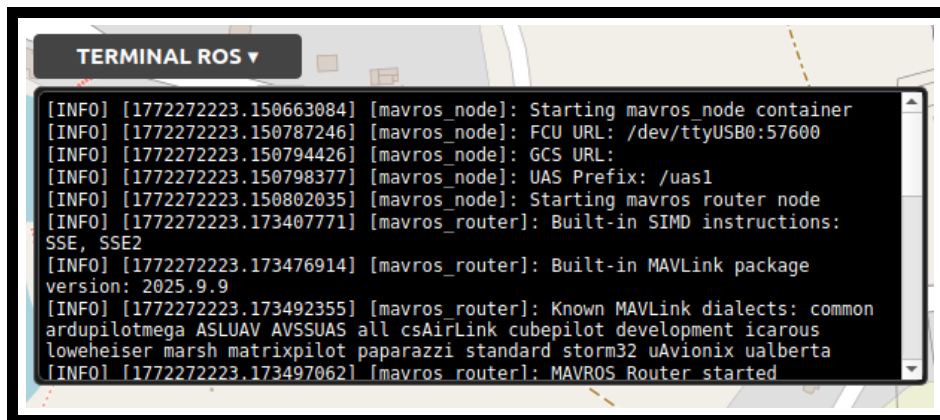
Sous le bouton permettant de changer de repère, on trouve un bouton indiquant le statut de l'armement du drone : "ARMEMENT UNKNOWN" sur fond gris si on ne le connaît pas, "ARMED" sur fond rouge si le drone est armé et "DISARMED" sur fond vert si le drone est désarmé. Cliquer sur ce bouton va permettre d'armer ou de désarmer le drone via le nœud ROS2 "mavros". Il est à noter que le bouton ne met à jour son affichage que lorsqu'il a reçu la confirmation via un message ROS2 que le changement de l'état d'armement du robot a bien été pris en compte.

Le bouton du dessous, quant à lui, permet d'afficher le mode du robot : "GUIDED MODE" sur fond bleu si le drone est en mode guidé, "MANUAL MODE" sur fond vert si le drone est en mode manuel et "UNKNOWN MODE" sur fond gris si l'on ne connaît pas le mode du robot où s'il est dans un autre mode. Cliquer sur ce bouton va permettre de faire passer le drone du mode manuel au mode guidé, toujours via le nœud

ROS "mavros". Ici aussi, le bouton ne met à jour son affichage que lorsqu'il a reçu la confirmation via un message ROS2 que le changement de mode a bien été pris en compte.

Le bouton "EMERGENCY" quant à lui, envoie une demande de désarmement du drone au nœud "mavros". Il a été rajouté afin que l'opérateur sache immédiatement où appuyer pour arrêter le drone (le bouton "ARMED" ferait également l'affaire mais il serait moins instinctif pour l'opérateur de penser à cliquer dessus en cas de problème).

Le bouton dépliant "MAVROS" permet à l'opérateur de choisir avec quel baudrate et sur quel port et sur quelle plateforme (rover ou intercepteur) lancer le nœud ROS2 "mavros". Les ports disponibles sont scannés au préalable. Cliquer sur le bouton "LAUNCH MAVROS" va créer une session screen dans laquelle on va lancer le nœud. Les messages qui seraient normalement affichés dans le terminal lorsqu'on lance "mavros" le sont dans la zone d'affichage dépliant volontairement similaire à un terminal en haut à droite de l'écran [Figure 38].



```
[INFO] [1772272223.150663084] [mavros_node]: Starting mavros node container
[INFO] [1772272223.150787246] [mavros_node]: FCU URL: /dev/ttyUSB0:57600
[INFO] [1772272223.150794426] [mavros_node]: GCS URL:
[INFO] [1772272223.150798377] [mavros_node]: UAS Prefix: /uas1
[INFO] [1772272223.150802035] [mavros_node]: Starting mavros router node
[INFO] [1772272223.173407771] [mavros_router]: Built-in SIMD instructions:
SSE, SSE2
[INFO] [1772272223.173476914] [mavros_router]: Built-in MAVLink package
version: 2025.9.9
[INFO] [1772272223.173492355] [mavros_router]: Known MAVLink dialects: common
ardupilotmega ASLUAV AVSSUAS all csAirLink cubepilot development icarous
loweheiser marsh matrixpilot paparazzi standard storm32 uAvionix ualberta
[INFO] [1772272223.173497062] [mavros_router]: MAVROS Router started
```

FIGURE 38 – "Terminal" où sont affichés les messages publiés par "mavros"

L'IHM n'a pas malheureusement pas pu être testée sur l'intercepteur mais seulement sur le rover où elle est parvenue à lancer "mavros" comme souhaité. Elle possède néanmoins quelques limites. Lancée sur le drone, l'affichage de cette dernière est transmis à l'opérateur via secure screen (SSH) avec l'option "-X". Elle reste donc très sensible à la qualité de la connexion Wi-Fi. De plus, si cette dernière s'interrompt entre l'ordinateur de l'opérateur et le drone, les fonctionnalités d'armement/désarmement du drone et d'arrêt d'urgence ne fonctionnent pas. Il a donc fallu faire en sorte de rajouter des mécanismes de sécurité additionnels, comme par exemple, le fait que les commandes envoyées par télécommandes soient prioritaires par rapport à celles envoyées par les programmes d'interception.

IV Validation de nos algorithmes sur le rover R1

Avant les essais en mer, le système a été testé sur un rover terrestre pour valider la loi de commande dans un environnement contrôlé. La cible est un cône de signa-

lisation jaune, détecté par segmentation couleur HSV (OpenCV). Ce choix permet de tester le contrôleur indépendamment de YOLO et de la plateforme marine.

Configuration

La détection OpenCV fournit des bounding boxes stables et propres. Le contrôleur IBVS utilisé est exactement le même que sur le bateau : même code, mêmes paramètres initiaux. Seul $V_{\max}$ a été adapté à la dynamique du rover.

Résultats



FIGURE 39 – Suivi du cône jaune fluo par asservissement visuel 2,5D.

Les résultats sont très satisfaisants. Le rover centre la cible rapidement et maintient la distance désirée de façon stable. L'erreur e_1 converge vers zéro en quelques secondes et e_2 reste proche de sa valeur désirée. Ces résultats ont confirmé que la loi

de commande fonctionne correctement et ont donné la confiance pour passer aux essais sur bateau.

V Validation de nos algorithmes sur l'intercepteur M1800

Après la validation sur rover, le système a été intégré sur l'USV intercepteur. La détection est assurée par YOLO avec le modèle Odessa, déployé via TensorRT sur la Jetson. Aucun de ces essais n'utilise ByteTrack, qui a été intégré ultérieurement.

Conditions et résultats

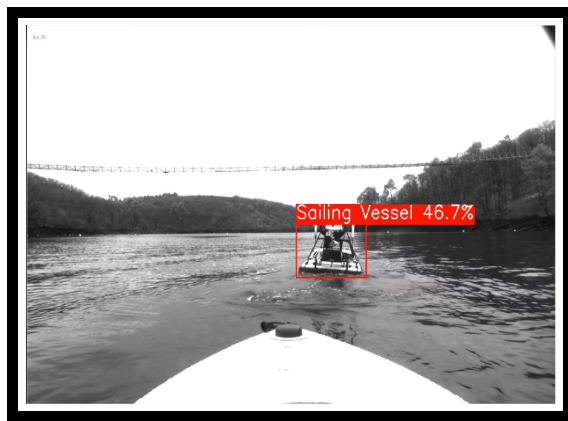


FIGURE 40 – Vue de la caméra monochrome avec la boîte d'ancrage déterminée par YOLO autour du bateau cible.



FIGURE 41 – Interception en cours

Les trois essais ont été réalisés dans les mêmes conditions : la caméra RGB principale n'était pas fonctionnelle, et le système a opéré uniquement avec une caméra monochrome. YOLO étant entraîné sur des images couleur, les détections obtenues étaient plus bruitées et moins stables que prévu [Figure 40]. Malgré cela, le protocole a été répété trois fois avec trois bateaux cibles différents, et les trois essais ont donné de très bons résultats. Le système a suivi et intercepté la cible dans les trois cas, sans modification du code ni des paramètres entre les essais. Le filtrage exponentiel (10) a bien compensé le bruit des détections, et le timeout a évité tout comportement non contrôlé lors des courtes pertes de détection. Ces résultats ont dépassé nos attentes et ont confirmé que l'asservissement visuel reste fonctionnel même dans des conditions non idéales.

Voici quelques liens vers les vidéos des essais :

- Détection d'une pame avec YOLO - 1
- Détection d'une pame avec YOLO - 2
- Détection d'une pame avec YOLO - 3
- Interception d'une pame - 1

- Interception d'une prame - 2
- Interception de l'USV cible - 1
- Interception de l'USV cible - 2

Conclusion

L'objectif du projet était de réaliser l'interception d'un USV cible. Pour ne pas abîmer le matériel, cette interception n'a pas été réalisée physiquement, mais nous nous sommes contentés de maintenir l'intercepteur derrière la cible. Nous avons montré qu'il est possible d'intercepter une embarcation mobile de façon autonome en utilisant uniquement une caméra. La loi de commande 2,5D, combinée à YOLO et au filtrage exponentiel, constitue un système fonctionnel et robuste. Les essais en conditions dégradées ont confirmé que le système tient même quand les capteurs principaux tombent en panne, ce qui est un résultat important pour des applications marines réelles.

L'algorithme de propagation ByteTrack a été intégré suite aux expérimentations pour améliorer la gestion des détections multiples. Son impact sur les performances reste à évaluer lors d'essais futurs.

D'autres améliorations sont également envisagées : l'évaluation du contrôleur NMPC en conditions réelles ainsi que l'ajout d'un comportement de recherche en cas de perte de cible.

Annexe

Lien vers le dépôt GitHub contenant les codes du projet :

- <https://github.com/Mathosmls/Interceptor/tree/main>